

Universidad de Castilla-La Mancha
Escuela Superior de Ingeniería Informática de Albacete

Trabajo Fin de Grado
Grado en Ingeniería Informática
Computación

Especificación de formato y desarrollo de herramientas para el procesamiento de currículum vitae en el ámbito académico universitario

Carlos Martínez Jaén

Julio, 2026

Escuela Superior de Ingeniería Informática Albacete

UCLM Universidad Castilla-La Mancha

TRABAJO FIN DE GRADO

Grado en Ingeniería Informática

Computación

Especificación de formato y desarrollo de herramientas para el procesamiento de currículum vitae en el ámbito académico universitario

Autor: Carlos Martínez Jaén

Director: Luis De La Ossa Jiménez

Codirector: José Antonio Gámez Martin

Julio, 2026

Aquí va la dedicatoria que cada cual quiera escribir. El ancho se controla manualmente

Declaración de autoría

Yo, Carlos Martínez Jaén , con DNI 00000000X , declaro que soy el único autor del trabajo fin de grado titulado " Especificación de formato y desarrollo de herramientas para el procesamiento de currículum vitae en el ámbito académico universitario ", que el citado trabajo no infringe las leyes en vigor sobre propiedad intelectual, y que todo el material no original contenido en dicho trabajo está apropiadamente atribuido a sus legítimos autores.

Albacete, a 25 de Julio de 2026

Fdo.: Carlos Martínez Jaén

Resumen

CVN constituye la norma española de referencia para representar información curricular de investigadores, mantenida por la Fundación Española para la Ciencia y la Tecnología (FECYT). Sin embargo, la existencia de este formato normalizado no elimina el coste de crear, mantener y adaptar currículos: su representación XML oficial y la herramienta oficial asociada no están pensadas como base directa de aplicaciones abiertas y ligeras, y el paquete oficial reúne fuentes heterogéneas (manuales, modelos técnicos y tablas auxiliares) cuyo uso automatizado exige un análisis previo.

Este Trabajo Fin de Grado presenta Open CVN, una arquitectura computacional abierta que separa la interoperabilidad estructural del significado curricular sin perder trazabilidad hacia las fuentes oficiales. El sistema genera de forma reproducible bindings estructurales a partir de los esquemas XSD del ecosistema CVN, normaliza los metadatos dispersos entre el manual funcional y el modelo en árbol, aplica una política semántica conservadora respaldada por evidencia concreta de cada tabla de valores controlados, y deriva de ella un modelo de dominio y un modelo conceptual agnóstico. Sobre esa base se define un formato de intercambio abierto, Open CVN JSON, junto con un parser, un validador y mecanismos de importación desde CVN XML y CVN PDF, con una vía determinista prioritaria y un recurso opcional a modelos de lenguaje de gran tamaño cuando esta no resulta suficiente. Una herramienta local de gestión, de línea de órdenes, demuestra el uso completo de esta representación: almacenamiento, versionado de currículos maestro y derivados, y exportación a JSON, LaTeX y PDF.

El sistema se evalúa mediante una batería de 488 pruebas automatizadas y reproducibles, organizada según las mismas capas de la arquitectura, que se ejecuta también en integración continua ante cada contribución propuesta. Los resultados confirman garantías fuertes allí donde el proceso es determinista (generación estructural, normalización, política semánti-

ca y validación del formato) y garantías parciales, documentadas de forma explícita, en la importación de CVN XML y en la importación asistida por modelos de lenguaje.

Agradecimientos

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Índice general

1 Introducción, motivación y objetivos 1

1.1 Contexto 1

1.2 Motivación 2

1.3 Objetivos 3

1.4 Alcance del proyecto 5

1.5 Competencias desarrolladas 6

1.6 Contribuciones 7

1.7 Estructura del documento 8

2 Antecedentes y estado del arte 11

2.1 Representación de información curricular compleja 11

2.2 Interoperabilidad curricular y sistemas de información de investigación 12

2.3 Formatos de serialización y validación 14

2.3.1 XML y XSD 14

2.3.2 JSON y JSON Schema 15

2.3.3 JSON-LD y YAML 15

2.4 Modelado conceptual y generación de artefactos 17

2.5 Síntesis del estado del arte 18

3 Análisis del ecosistema CVN y propuesta de solución 21

3.1 El paquete oficial CVN como fuente del proyecto 21

3.2 El XML embebido en los documentos CVN 24

3.3 Limitaciones y dificultades técnicas detectadas 24

3.4 Propuesta de solución: la arquitectura Open CVN 27

3.5 Alcance y exclusiones 29

4 Metodología, herramientas y arquitectura general 31

4.1 Metodología de trabajo 31

4.2 Herramientas empleadas 32

4.3 Arquitectura general por capas 35

4.3.1 Fuente oficial CVN 35

4.3.2 Generación estructural y normalización 35

4.3.3 Dominio, modelo conceptual y formato 36

4.3.4 Herramienta local 37

4.4 Contratos entre capas y razonamiento arquitectónico 38

5 Implementación del pipeline de generación y normalización 39

5.1 Generación estructural 39

5.2 Normalización de metadatos por código CVN 40

5.3 Resolución de referencias auxiliares 40

5.4 Política semántica 41

5.4.1 Formas semánticas reconocidas 42

5.4.2 Enumeración cerrada frente a catálogo abierto 43

5.4.3 Presencia, nomenclatura y traza 43

5.5 Generación de modelos de dominio y del modelo conceptual 43

5.6 Generación de JSON Schema y trazabilidad 44

5.7 Síntesis de las etapas del pipeline 45

6 Formato Open CVN, validación y herramienta de gestión 47

6.1 El formato Open CVN JSON 47

6.2 Parser y validador 48

6.3 Importación de currículos existentes 50

6.4 Herramienta local de gestión 50

6.5 Alcance funcional y garantías 53

7 Evaluación, resultados y discusión 55

7.1 Diseño de la evaluación 55

7.2 Niveles de evaluación 56

7.3 Comando de verificación y resultados 57

7.4 Discusión de garantías y limitaciones 57

8 Conclusiones, competencias y trabajo futuro 61
8.1 Resumen del trabajo 61
8.2 Cumplimiento de los objetivos 62
8.3 Contribuciones principales 63
8.4 Competencias desarrolladas 64
8.5 Limitaciones y trabajo futuro 65
8.6 Conclusiones 67
Referencia bibliográfica 71
Declaración de uso de Inteligencia Artificial 73
Anexos 75
Anexo A: Instalación, entorno y comandos principales 77
A.1 Prerrequisitos y configuración del entorno 77
A.2 Regeneración de artefactos y verificación 78
A.3 Recorrido completo con la herramienta local 78
Anexo B: Estructura completa del formato Open CVN JSON 81
B.1 Bloque `metadata` 81
B.2 Bloque `curriculum` 81
B.3 Valores de referencia controlada 82
B.4 Definiciones del esquema 83
Anexo C: Desarrollo del proyecto por fases técnicas 85
C.1 Fase 1: generación estructural y normalización 85
C.2 Fase 2: modelo conceptual, formato Open CVN y parser 86
C.3 Fase 3: aplicación de gestión local 87
C.4 Correcciones de alcance 87
Anexo D: Repositorio del proyecto y guía de uso 89

Índice de figuras

3.1 Separación entre el paquete oficial CVN y la arquitectura Open CVN. . . . 27
3.2 Vista UML compacta del esquema conceptual de Open CVN, con el número de conceptos identificados por área curricular. 28

4.1 Arquitectura general de Open CVN organizada por capas, con el módulo del repositorio responsable de cada bloque. 35

5.1 Trazabilidad de un campo desde el paquete oficial CVN hasta el esquema Open CVN JSON. 45

6.1 Flujo de importación y validación de currículos hacia Open CVN. 49

Índice de tablas

1.1 Relación entre objetivos específicos y capítulos del documento. 5

2.1 Comparativa de modelos de interoperabilidad curricular relacionados con Open CVN. 13
2.2 Comparativa de formatos de serialización y validación relevantes para Open CVN. 17

3.1 Fuentes del paquete oficial CVN analizadas y su uso en el proyecto. 23
3.2 Limitaciones detectadas en el paquete oficial CVN y respuesta de diseño de Open CVN. 26

4.1 Herramientas empleadas en el proyecto y justificación de su uso. 34
4.2 Módulos del repositorio y responsabilidades dentro de la arquitectura Open CVN. 37

5.1 Resultado de la resolución de referencias auxiliares sobre las entradas normalizadas con tabla asociada. 41
5.2 Formas semánticas reconocidas por la política semántica. 42
5.3 Etapas del pipeline de generación y normalización, entrada, salida y módulo responsable. 46

6.1 Comandos principales de la herramienta local de gestión. 51

7.1 Categorías de pruebas automatizadas y objetivo de cada una. 56
7.2 Garantías del sistema frente a limitaciones conocidas. 58

8.1 Grado de cumplimiento de los objetivos específicos. 63
8.2 Competencias de Computación y evidencia concreta de cada una. 64
8.3 Limitaciones documentadas y trabajo futuro asociado. 67
B.1 Envoltura común de las entradas de las listas curriculares. 82
B.2 Distribución de las definiciones del esquema Open CVN JSON por área conceptual. 83

C.1 Unidades de trabajo de la Fase 1, agrupadas bajo el epic de automatización de la traducción CVN XML/XSD. 86
C.2 Unidades de trabajo de la Fase 2, agrupadas bajo el epic de esquema agnóstico, JSON Schema y parser. 86
C.3 Unidades de trabajo de la Fase 3, agrupadas bajo el epic de la aplicación de gestión de currículos. 87
C.4 Correcciones de alcance aplicadas durante el desarrollo. 88

Índice de algoritmos

Índice de listados de código

5.1 Fragmento reducido del metadato normalizado del campo de sexo. 40

6.1 Estructura raíz mínima de un documento Open CVN JSON. 48
6.2 Currículo maestro reducido, con una entrada de investigación. 52
6.3 Versión derivada “publica” del mismo currículo, con la entrada de investigación excluida. 52

7.1 Comando principal de verificación del sistema. 57

A.1 Preparación del entorno de desarrollo. 78
A.2 Regeneración de artefactos estructurales y verificación por pruebas. 78
A.3 Recorrido completo de importación, versionado y exportación con la herramienta local. 79

1. Introducción, motivación y objetivos

El presente capítulo introduce el contexto general en el que se enmarca este Trabajo Fin de Grado, describe la motivación que justifica su desarrollo y delimita los objetivos que guían la solución propuesta. Para ello, en primer lugar se analiza el papel del Currículum Vítae Normalizado (CVN) en la gestión de información académica e investigadora en España. A continuación, se expone la necesidad de disponer de una representación de bajo nivel que facilite el procesamiento automático de los datos curriculares. Finalmente, se establecen el alcance del proyecto, las competencias de Computación trabajadas, las principales contribuciones realizadas y la estructura seguida por el resto del documento.

1.1. Contexto

La actividad académica e investigadora produce una cantidad considerable de información curricular de naturaleza heterogénea. En un mismo currículo pueden aparecer datos relativos a formación, experiencia profesional, publicaciones, proyectos de investigación, contratos, estancias, dirección de trabajos, transferencia, actividad docente y otros méritos que responden a estructuras y criterios de descripción diferentes. Esta información no se utiliza únicamente como documento de presentación personal, sino también como entrada para procesos de evaluación, convocatorias públicas, solicitudes de ayudas, acreditaciones, portales institucionales y sistemas internos de gestión de la investigación. En consecuencia, el valor de un currículo no depende solo de su apariencia final, sino también de la capacidad de los sistemas informáticos para interpretar, validar, reutilizar y transformar los datos que contiene.

En el ámbito español, el Currículum Vítae Normalizado (CVN) constituye una norma de referencia para la presentación de información curricular de investigadores. La Fundación Española para la Ciencia y la Tecnología (FECYT) lo define como una norma estándar que

establece un formato común de presentación de los datos curriculares y facilita la interoperabilidad con bases de datos institucionales [FECYT, 2026]. Esta función normalizadora resulta especialmente relevante en un entorno en el que una misma trayectoria académica puede tener que presentarse ante organismos, plataformas y convocatorias con requisitos distintos. Por ello, CVN representa un punto de partida adecuado para estudiar cómo puede formalizarse la información curricular de forma computable y reutilizable.

No obstante, la existencia de un formato normalizado no elimina por sí misma el coste asociado a la elaboración, el mantenimiento y la adaptación de los currículos. A pesar de disponer de una referencia común, los profesores e investigadores universitarios continúan dedicando un tiempo significativo a actualizar méritos, preparar versiones específicas, adaptar contenidos a formatos concretos o trasladar información entre herramientas. Esta situación se ve agravada cuando el mismo historial debe reutilizarse en contextos diferentes, ya que pequeñas variaciones en la estructura, el nivel de detalle o el formato de salida pueden obligar a realizar tareas manuales repetitivas. Desde esta perspectiva, resulta necesario avanzar hacia herramientas abiertas que automaticen parte del proceso y que permitan gestionar los currículos de manera más sistemática.

El tratamiento computacional de CVN, sin embargo, no puede reducirse a la generación de un documento final ni a la lectura de un único fichero XML. La base del trabajo es la norma CVN y el ecosistema de artefactos que la acompaña; dentro de dicho ecosistema, XML actúa como mecanismo de representación e intercambio de datos. En particular, el paquete oficial analizado en este trabajo incluye esquemas XML/XSD, manuales estructurados, modelos auxiliares y tablas de referencia que describen la estructura de los datos, sus códigos internos y su organización jerárquica. Estos elementos aportan la base formal necesaria para la interoperabilidad, pero también introducen una complejidad que dificulta su uso directo como modelo interno de herramientas abiertas, ligeras y reproducibles. En conjunto, el problema abordado consiste en separar el significado de la información curricular de la forma concreta en que la norma la representa y la serializa en sus artefactos técnicos.

1.2. Motivación

La motivación principal de este proyecto surge de la necesidad de definir una representación de bajo nivel para currículos académicos e investigadores basada en la norma CVN, pero orientada al procesamiento automático de datos. Dicha representación debe permitir que la información curricular pueda almacenarse, validarse, transformarse y exportarse sin depender exclusivamente de la representación XML empleada por CVN ni de la herramienta oficial

asociada. Esta necesidad está directamente relacionada con la posibilidad de desarrollar herramientas abiertas para la creación, actualización y adaptación de currículos, tal y como se plantea en la descripción oficial del Trabajo Fin de Grado.

Desde el punto de vista técnico, el uso directo de los artefactos oficiales de CVN como modelo interno presenta varias dificultades. Los esquemas XSD proporcionan una descripción estructural precisa de la representación XML, pero no siempre expresan de forma explícita la semántica curricular necesaria para construir aplicaciones de alto nivel. Además, parte de la información relevante se encuentra distribuida entre manuales, modelos auxiliares y tablas de referencia, lo que obliga a combinar varias fuentes para interpretar correctamente los campos, códigos y relaciones del dominio. Esta dispersión no invalida la norma CVN ni sus artefactos oficiales, que cumplen una función esencial de interoperabilidad, pero sí condiciona su reutilización como base directa para herramientas de gestión, validación o exportación.

El presente Trabajo Fin de Grado aborda este problema desde una perspectiva de Computación. La cuestión central no consiste únicamente en leer o escribir ficheros, sino en analizar fuentes formales heterogéneas, extraer evidencia estructural y semántica, normalizar metadatos, preservar trazabilidad y generar modelos capaces de representar información curricular compleja. De este modo, el proyecto se sitúa en el ámbito de la formalización y representación computable del conocimiento, con especial atención a la separación entre formato de serialización, modelo conceptual y artefactos de validación.

Como respuesta a esta situación, Open CVN propone una base técnica abierta, extensible y potencialmente colaborativa para representar currículos académicos tomando CVN como referencia. La solución no pretende sustituir la norma oficial, sino construir sobre ella una capa intermedia que facilite el desarrollo de herramientas posteriores. Esta capa debe permitir trabajar con un formato final basado en JSON, adecuado tanto para almacenamiento en archivos de texto como para su integración en bases de datos documentales o aplicaciones de procesamiento. De este modo, la norma CVN y sus artefactos oficiales se mantienen como fuentes de trazabilidad e interoperabilidad, mientras que el formato Open CVN se orienta a facilitar la gestión automatizada y la evolución controlada de los datos curriculares sin quedar limitado a la representación XML ni a la herramienta oficial existente.

1.3. Objetivos

El objetivo general de este Trabajo Fin de Grado es *diseñar e implementar una arquitectura computacional abierta para representar, validar, transformar, almacenar y exportar currículos académicos e investigadores tomando CVN como norma y ecosistema de referen-*

cia, pero evitando que su representación XML o la herramienta oficial asociada condicionen por completo el modelo interno del sistema, con el fin de facilitar el desarrollo de herramientas abiertas para la gestión automatizada de currículos.

Para alcanzar esta meta global se han definido los siguientes objetivos específicos:

1. **Analizar el ecosistema CVN y sus artefactos oficiales.** Este objetivo comprende el estudio de los esquemas XML/XSD, manuales estructurados, modelos auxiliares y tablas de referencia que forman parte del paquete empleado como fuente del proyecto.
2. **Estudiar alternativas de representación y validación de datos curriculares.** Se consideran modelos conceptuales, formatos de serialización y herramientas de generación de modelos con el fin de seleccionar una estrategia adecuada para información curricular compleja.
3. **Generar una capa estructural reproducible a partir de los XSD oficiales.** La generación automática permite obtener artefactos consistentes con la representación XML definida dentro del ecosistema CVN y, al mismo tiempo, separar el código generado de la lógica mantenida manualmente.
4. **Normalizar metadatos procedentes de las fuentes oficiales.** Este objetivo busca unificar la información funcional y técnica asociada a los códigos CVN, reduciendo la dispersión existente entre documentos auxiliares.
5. **Definir reglas semánticas conservadoras y trazables.** Las reglas deben permitir interpretar campos, tipos, tablas de referencia y relaciones sin perder la conexión con las fuentes originales de las que se derivan.
6. **Generar modelos de dominio y artefactos conceptuales.** Estos modelos deben representar la información curricular de forma más próxima al dominio académico que a la estructura concreta de la serialización XML.
7. **Definir un formato Open CVN JSON.** El formato propuesto debe ser validable, extensible y adecuado para intercambio, almacenamiento, colaboración y transformación posterior.
8. **Implementar mecanismos de parseo, validación, almacenamiento y exportación.** Estos mecanismos permiten demostrar que la representación definida puede emplearse en flujos reales de gestión curricular y generación de artefactos de salida.
9. **Diseñar mecanismos de importación deterministas y asistidos por modelos de lenguaje.** El objetivo consiste en incorporar a Open CVN currículos ya existentes en CVN

XML o en CVN PDF, priorizando siempre una vía de extracción determinista y reservando el uso de modelos de lenguaje de gran tamaño (del inglés, *Large Language Models*, en adelante LLM) como alternativa opcional para los casos en que la vía determinista no resulta suficiente. Esta vía asistida debe activarse de forma explícita, mantenerse trazable y quedar sujeta a validación posterior frente al formato Open CVN antes de aceptarse como resultado válido.

10. **Verificar el sistema mediante pruebas reproducibles.** La evaluación se apoya en pruebas automatizadas, comandos documentados y flujos de validación extremo a extremo, con el fin de establecer las garantías reales del sistema desarrollado.

La Tabla 1.1 relaciona estos objetivos con los capítulos principales en los que se desarrollan.

Objetivo	Descripción resumida	Capítulos
OE1	Análisis del ecosistema CVN y sus artefactos oficiales	2, 3
OE2	Estudio de alternativas de representación, modelado y validación	2
OE3	Generación estructural reproducible desde XSD	4, 5
OE4	Normalización de metadatos CVN	5
OE5	Definición de reglas semánticas trazables	3, 5
OE6	Generación de modelos de dominio y artefactos conceptuales	5
OE7	Definición del formato Open CVN JSON	6
OE8	Parser, validador, almacenamiento y exportación	6
OE9	Importación determinista y asistida por LLM	6
OE10	Verificación automatizada y flujos reproducibles	7

Tabla 1.1: Relación entre objetivos específicos y capítulos del documento.

1.4. Alcance del proyecto

El alcance del proyecto se centra en la construcción de una base computacional para trabajar con información curricular estructurada. El trabajo incluye el análisis del paquete oficial CVN empleado como fuente, la generación de bindings estructurales, la normalización de metadatos, la definición de reglas semánticas, la generación de modelos de dominio,

la especificación de un formato JSON propio y la implementación de herramientas de validación, importación, almacenamiento y exportación. Estas tareas se han organizado de forma que cada capa conserve trazabilidad hacia las fuentes originales y pueda verificarse mediante procedimientos reproducibles.

Dentro del alcance se incluye también la importación de currículos ya existentes hacia Open CVN a partir de CVN XML y de CVN PDF. Esta importación se resuelve en primer lugar mediante extracción determinista y solo recurre, de forma opcional y explícitamente autorizada, a modelos de lenguaje (LLM) cuando la vía determinista no permite recuperar información suficiente. El resultado de esta vía asistida no se presenta como una validación semántica completa del currículo importado, sino como una propuesta que debe superar la validación del formato Open CVN y quedar disponible para su revisión antes de considerarse definitiva.

Queda fuera del alcance garantizar una conversión completa y sin pérdida de todos los currículos CVN existentes, ya que dicha garantía requeriría cubrir la totalidad de variantes, casos particulares y restricciones prácticas del ecosistema oficial. Tampoco se pretende sustituir la norma CVN ni la infraestructura proporcionada por FECYT. La propuesta debe entenderse como una capa abierta y reproducible que facilita el tratamiento de datos curriculares cuando se necesita validación, transformación, almacenamiento local o exportación a otros formatos.

La herramienta local desarrollada dentro del proyecto cumple una función demostrativa y operativa. Su finalidad es comprobar que el formato propuesto y las capas de validación pueden utilizarse en flujos de importación, almacenamiento, versionado y exportación. No obstante, la contribución principal del TFG no reside en la interfaz de dicha herramienta, sino en la arquitectura de representación, normalización y transformación que permite construirla.

1.5. Competencias desarrolladas

El desarrollo del proyecto se alinea con competencias propias de la intensificación de Computación. En particular, de acuerdo con la descripción oficial del trabajo, se abordan las siguientes:

- **[CM1]** Capacidad para tener un conocimiento profundo de los principios fundamentales y modelos de la computación y saberlos aplicar para interpretar, seleccionar, valorar, modelar y crear nuevos conceptos, teorías, usos y desarrollos tecnológicos relacionados con la informática.

- **[CM2]** Capacidad para conocer los fundamentos teóricos de los lenguajes de programación y las técnicas de procesamiento léxico, sintáctico y semántico asociadas, y saber aplicarlas para la creación, diseño y procesamiento de lenguajes.
- **[CM5]** Capacidad para adquirir, obtener, formalizar y representar el conocimiento humano en una forma computable para la resolución de problemas mediante un sistema informático en cualquier ámbito de aplicación, particularmente los relacionados con aspectos de computación, percepción y actuación en ambientes o entornos inteligentes.
- **[CM6]** Capacidad para desarrollar y evaluar sistemas interactivos y de presentación de información compleja y su aplicación a la resolución de problemas de diseño de interacción persona-computadora.

Estas competencias se reflejan en el análisis de estructuras oficiales complejas, la formalización de información curricular en modelos computables, la generación automática de artefactos, la definición de reglas de validación, la construcción de una herramienta de gestión y la evaluación técnica del sistema mediante pruebas reproducibles. Su grado de cumplimiento se retomará en el capítulo de conclusiones, una vez presentados el desarrollo y los resultados obtenidos.

1.6. Contribuciones

Las principales contribuciones del trabajo pueden resumirse en los siguientes elementos:

- **Análisis computacional del ecosistema CVN:** se identifican las fuentes estructurales, funcionales y auxiliares necesarias para interpretar la información curricular representada en CVN, así como las restricciones que condicionan su uso automatizado.
- **Arquitectura reproducible por capas:** se separan los bindings generados, la normalización de metadatos, la política semántica, los modelos de dominio, el modelo conceptual, el formato JSON, la validación y la herramienta local, evitando que una única representación concentre todas las responsabilidades del sistema.
- **Trazabilidad entre CVN y Open CVN:** se conserva evidencia sobre códigos, fuentes, rutas y decisiones semánticas, con el fin de que la transformación hacia el nuevo formato mantenga una relación verificable con el estándar original.
- **Formato Open CVN JSON:** se define una representación abierta, validable y extensible orientada al intercambio, almacenamiento y exportación de datos curriculares.

- **Herramientas de validación y uso:** se implementan parsers, validadores, almacenamiento local, versionado curricular y exportación a otros formatos, lo que permite comprobar la utilidad práctica de la representación definida.
- **Importación determinista y asistida por LLM:** se implementan vías de importación desde CVN XML y CVN PDF hacia Open CVN, priorizando la extracción determinista y reservando el uso de modelos de lenguaje de gran tamaño como alternativa opcional, trazable y sujeta a validación posterior para los casos en que la extracción determinista resulta insuficiente.
- **Verificación automatizada:** se incorporan pruebas y flujos reproducibles para evaluar la consistencia de los artefactos generados y el comportamiento de las capas principales del sistema.

En conjunto, estas contribuciones no deben entenderse como una sustitución del ecosistema CVN, sino como una propuesta complementaria que facilita su tratamiento computacional en contextos donde resultan necesarias la validación formal, la transformación entre formatos y la gestión automatizada de currículos.

1.7. Estructura del documento

El resto del documento se organiza de forma progresiva, desde los antecedentes y el análisis del dominio hasta la implementación, evaluación y discusión de la solución desarrollada:

- **Capítulo 2: Antecedentes y estado del arte.** Presenta el contexto técnico y conceptual necesario para comprender el problema, incluyendo CVN, la interoperabilidad curricular, los modelos relacionados, los formatos de serialización y las alternativas de modelado.
- **Capítulo 3: Análisis del ecosistema CVN y propuesta de solución.** Describe las fuentes oficiales utilizadas, sus limitaciones y la propuesta de arquitectura Open CVN derivada de dicho análisis.
- **Capítulo 4: Metodología, herramientas y arquitectura general.** Explica el proceso técnico seguido, las herramientas empleadas y la organización por capas del sistema.
- **Capítulo 5: Implementación del *pipeline* de generación y normalización.** Detalla la generación estructural, la normalización de metadatos, la política semántica y la generación de modelos y artefactos conceptuales.

- **Capítulo 6: Formato Open CVN, validación y herramienta de gestión.** Presenta el formato JSON propuesto, los parsers, validadores, las vías de importación determinista y asistida por LLM, los mecanismos de almacenamiento local y los flujos de exportación.
- **Capítulo 7: Evaluación, resultados y discusión.** Describe la estrategia de verificación, los resultados obtenidos y las garantías reales que ofrece el sistema desarrollado.
- **Capítulo 8: Conclusiones, competencias y trabajo futuro.** Resume el grado de cumplimiento de los objetivos, relaciona el trabajo con las competencias desarrolladas, recoge las limitaciones identificadas y plantea líneas futuras.

2. Antecedentes y estado del arte

El presente capítulo recoge los antecedentes técnicos que sustentan el trabajo: interoperabilidad de información investigadora, formatos de serialización, mecanismos de validación y técnicas de modelado conceptual. Su finalidad es comparar alternativas y justificar las decisiones tecnológicas que se utilizarán en los capítulos posteriores, sin desarrollar todavía el análisis específico del dominio CVN. Ese análisis se reserva para el capítulo siguiente, donde se estudiarán sus artefactos oficiales, sus limitaciones y la propuesta Open CVN construida a partir de ellos.

2.1. Representación de información curricular compleja

La información curricular académica e investigadora constituye un dominio de datos heterogéneo, con entidades, relaciones, restricciones y niveles de detalle que no encajan bien en una representación plana. Desde una perspectiva computacional, esta circunstancia obliga a diferenciar el modelo conceptual del dominio, la serialización utilizada para intercambiar datos y las herramientas que permiten procesarlos. Esta separación será una idea recurrente en el resto del documento, porque permite razonar sobre el significado de los datos sin confundirlo con el formato concreto en el que se almacenan o transmiten.

En España, CVN proporciona una referencia normalizada para la presentación de información curricular de investigadores y favorece la interoperabilidad con bases de datos institucionales [FECYT, 2026]. En este capítulo se menciona únicamente como punto de conexión con el dominio de aplicación. La descripción de sus fuentes, de su representación técnica y de sus limitaciones se reserva para el capítulo siguiente, con el fin de no anticipar el análisis que corresponde a la propuesta de solución.

2.2. Interoperabilidad curricular y sistemas de información de investigación

La gestión de información curricular no es un problema exclusivo de CVN. En el ámbito académico internacional existen diferentes iniciativas orientadas a mejorar la interoperabilidad entre sistemas de información de investigación. Estos sistemas, conocidos habitualmente como *Current Research Information Systems* (CRIS), integran datos sobre investigadores, organizaciones, proyectos, publicaciones, financiación, resultados científicos y otros elementos relevantes para la actividad investigadora.

La interoperabilidad en este dominio no depende solo de compartir documentos, sino también de poder identificar de forma estable a las personas, organizaciones, resultados y relaciones descritas. ORCID, por ejemplo, proporciona identificadores persistentes para investigadores y mecanismos de conexión entre dichos identificadores, contribuciones y afiliaciones [ORCID, 2026]. En una línea complementaria, RO-Crate propone un enfoque ligero para empaquetar datos de investigación junto con sus metadatos [RO-Crate Community, 2024]. Estas iniciativas no resuelven por sí mismas la representación completa de un currículo CVN, pero muestran la importancia de combinar datos estructurados, metadatos explícitos e identificadores reutilizables.

Uno de los modelos más representativos en este ámbito es CERIF, mantenido por euroCRIS. CERIF se presenta como un formato europeo para la gestión, intercambio y almacenamiento de información de investigación, con especial atención a la interoperabilidad entre sistemas CRIS [euroCRIS, 2026]. Su principal ventaja reside en la riqueza de su modelo, que permite representar entidades, relaciones y dimensiones temporales con un alto grado de precisión. Esta expresividad resulta adecuada para infraestructuras institucionales complejas, aunque también introduce un nivel de abstracción y complejidad que puede ser excesivo cuando el objetivo es definir una representación ligera y directamente procesable de currículos.

Otra aproximación relevante es VIVO, una plataforma orientada a conectar, compartir y descubrir información sobre investigación mediante tecnologías de Web Semántica [VIVO Project, 2026]. Frente a modelos más tabulares o documentales, VIVO se apoya en grafos y ontologías para describir investigadores, publicaciones, organizaciones y relaciones académicas. Esta orientación facilita la publicación de datos enlazados y la integración con vocabularios semánticos, pero exige una infraestructura conceptual y tecnológica más compleja que la requerida por una herramienta local de gestión curricular.

En el contexto español destaca también el proyecto HERCULES y, dentro de él, la

Research Ontology for Hercules (ROH). La red de ontologías HERCULES persigue la representación semántica de información de investigación en el sistema universitario español [Universidad de Murcia, 2026, Proyecto HERCULES, 2026]. Además, parte de su ecosistema se ha relacionado con CVN mediante código y artefactos públicos orientados a la importación de información curricular [Deusto HERCULES, 2026b, Deusto HERCULES, 2026a]. Esta proximidad al dominio español convierte a ROH en una referencia relevante, aunque su orientación ontológica y su dependencia de tecnologías de Web Semántica hacen que no sea la opción más sencilla para el alcance de este trabajo.

La Tabla 2.1 resume estos tres enfoques atendiendo al tipo de representación empleada, su principal fortaleza y la limitación que presenta cada uno frente al objetivo de Open CVN.

Modelo	**Enfoque de representación**	**Fortaleza principal**	**Limitación frente a Open CVN**
CERIF	Modelo europeo orientado a entidades y relaciones para sistemas CRIS	Alta expresividad para relaciones institucionales y dimensiones temporales	Complejidad y coste de adopción elevados para una representación ligera
VIVO	Grafos y ontologías de Web Semántica	Publicación de datos enlazados e integración con vocabularios semánticos	Requiere infraestructura RDF/ontológica no necesaria para el alcance del TFG
ROH	Ontología del ámbito universitario e investigador español (proyecto HERCULES)	Proximidad al dominio CVN español y artefactos de importación ya existentes	Orientación ontológica y dependencia de tecnologías de Web Semántica

Tabla 2.1: Comparativa de modelos de interoperabilidad curricular relacionados con Open CVN.

En conjunto, CERIF, VIVO y ROH muestran que la información investigadora puede modelarse con niveles elevados de riqueza semántica. No obstante, también ponen de manifiesto una tensión habitual entre expresividad e implementabilidad. Cuanto más completo y semánticamente rico es un modelo, mayor suele ser el coste de adopción, mantenimiento, consulta y validación. Para Open CVN, esta observación resulta determinante: el objetivo no es competir con infraestructuras CRIS ni con ontologías de investigación completas, sino definir una representación curricular abierta, validable y suficientemente expresiva para el ámbito del TFG.

2.3. Formatos de serialización y validación

La elección de un formato de serialización condiciona la forma en que los datos se almacenan, intercambian y procesan. Sin embargo, un formato de serialización no equivale por sí mismo a un modelo conceptual. Un mismo dominio puede representarse en XML, JSON, YAML u otros formatos, siempre que exista una definición clara de las entidades, atributos, relaciones y restricciones que deben preservarse. Por ello, en este trabajo los formatos se analizan como mecanismos técnicos, no como la única fuente de significado del sistema.

2.3.1. XML y XSD

Extensible Markup Language (XML) es un lenguaje de marcado definido por el World Wide Web Consortium y ampliamente utilizado para representar documentos y datos estructurados. La recomendación XML 1.0 lo define como un perfil de SGML diseñado para facilitar el intercambio y procesamiento de documentos en la Web, con atención explícita a la interoperabilidad y a la facilidad de implementación de procesadores [World Wide Web Consortium, 2008]. XSD se define, a su vez, como un lenguaje para describir la estructura y restringir el contenido de documentos XML, aunque la propia especificación reconoce que algunas aplicaciones pueden necesitar validaciones adicionales no expresables únicamente mediante el esquema [World Wide Web Consortium, 2012].

La principal ventaja de XML en este contexto es su capacidad para representar estructuras jerárquicas complejas y asociarlas a esquemas de validación robustos. Esto permite comprobar si un documento respeta una estructura determinada y facilita la interoperabilidad entre sistemas que comparten la misma definición. Además, el uso de XSD proporciona una fuente formal a partir de la cual pueden generarse artefactos estructurales, como clases de datos o bindings de acceso programático.

No obstante, XML también presenta limitaciones cuando se utiliza como base directa de una herramienta de gestión. Su sintaxis es más verbosa que la de formatos más ligeros, el procesamiento suele requerir bibliotecas específicas y la estructura del documento no siempre coincide con el modelo conceptual más adecuado para una aplicación. Por tanto, XML puede ser una fuente técnica fundamental en sistemas de intercambio estructurado, pero no constituye por sí solo el modelo completo del dominio representado.

2.3.2. JSON y JSON Schema

JavaScript Object Notation (JSON) es un formato ligero de intercambio de datos basado en estructuras de pares clave-valor y listas ordenadas [JSON.org, 2026]. Su sencillez sintáctica, su integración directa con lenguajes de programación y su uso extendido en servicios web lo convierten en una alternativa adecuada para representaciones orientadas al procesamiento automático. En Python, por ejemplo, un documento JSON puede mapearse de forma natural a diccionarios, listas y tipos primitivos, lo que simplifica su manipulación en aplicaciones.

La principal dificultad de JSON es que, por sí solo, no define restricciones estructurales ni semánticas. Para resolver esta limitación se utiliza JSON Schema, un vocabulario que permite describir la estructura esperada de documentos JSON, incluyendo tipos, campos obligatorios, restricciones de valores y composición de objetos. La documentación de JSON Schema lo presenta como un lenguaje declarativo para definir estructura y restricciones de datos JSON, cuya validación requiere comprobar una instancia contra un esquema mediante herramientas específicas [JSON Schema, 2026]. De este modo, JSON puede combinarse con un mecanismo de validación formal similar en propósito a XSD, aunque con una sintaxis más cercana a los formatos habituales de intercambio de datos en aplicaciones modernas.

En el ecosistema Python, Pydantic complementa esta aproximación al permitir definir modelos mediante anotaciones de tipo, validar datos de entrada, serializar estructuras y emitir JSON Schema a partir de los modelos [Pydantic, 2026]. Esta combinación resulta especialmente adecuada para un proyecto que necesita mantener correspondencia entre un formato de intercambio, reglas de validación y objetos manipulables desde código. Al mismo tiempo, obliga a distinguir entre la validación sintáctica o estructural, que puede comprobarse de forma automática, y las reglas semánticas del dominio, que requieren análisis adicional de las fuentes CVN.

Para el contexto de Open CVN, JSON resulta especialmente adecuado como formato final de intercambio y almacenamiento porque permite construir documentos legibles, validables y compatibles con herramientas habituales de desarrollo. Esta elección no implica renunciar a una referencia normativa del dominio. Al contrario, el objetivo consiste en derivar una representación JSON trazable a partir de fuentes oficiales, preservando la relación entre los datos, sus reglas y su interpretación curricular.

2.3.3. JSON-LD y YAML

JSON for Linking Data (JSON-LD) extiende JSON para facilitar la representación de datos enlazados mediante contextos semánticos. La recomendación JSON-LD 1.1 lo de-

fine como una serialización basada en JSON para *Linked Data*, pensada para integrarse con sistemas que ya utilizan JSON y facilitar un camino de evolución hacia datos enlazados [World Wide Web Consortium, 2020]. Su principal aportación consiste en permitir que un documento JSON pueda interpretarse también como parte de un grafo de conocimiento, asociando claves del documento con identificadores semánticos. Esta característica lo convierte en una opción interesante cuando se busca compatibilidad con tecnologías RDF u ontologías.

Sin embargo, la incorporación de semántica enlazada también aumenta la complejidad del sistema. Para un proyecto cuyo objetivo principal es definir una representación curricular abierta, validable y manejable mediante herramientas Python, JSON-LD puede resultar más expresivo de lo necesario en una primera aproximación. Su interés se mantiene como posible línea de evolución futura, en especial si Open CVN necesitara integrarse con ontologías como ROH o con infraestructuras de datos enlazados.

YAML, por su parte, es un formato orientado a la legibilidad humana y empleado con frecuencia en configuración y definición de esquemas [YAML.org, 2026]. Su sintaxis compacta facilita la lectura manual, aunque puede introducir ambigüedades y requiere parsers específicos. En el contexto de este trabajo, YAML resulta relevante principalmente por su uso en herramientas de modelado como LinkML, pero no se adopta como formato final de intercambio curricular.

La Tabla 2.2 recoge, a modo de síntesis, la función principal, el mecanismo de validación asociado y el papel que cada uno de estos formatos desempeña dentro de Open CVN.

Formato	Función principal	Validación asociada	Papel en Open CVN
XML + XSD	Representación jerárquica de documentos y datos estructurados	Esquema XSD	Fuente estructural oficial de CVN; no se adopta como formato final de intercambio
JSON + JSON Schema	Intercambio ligero de datos orientado a aplicaciones	JSON Schema, con apoyo de Pydantic en Python	Formato final de intercambio, almacenamiento y validación de Open CVN
JSON-LD	Extensión de JSON para datos enlazados semánticamente	Contextos JSON-LD y vocabularios semánticos	Línea de evolución futura; no se adopta en la versión actual
YAML	Formato legible orientado a configuración y definición de esquemas	Depende de la herramienta que lo consume (por ejemplo, LinkML)	Referencia en herramientas de modelado; no se emplea como formato de intercambio final

Tabla 2.2: Comparativa de formatos de serialización y validación relevantes para Open CVN.

2.4. Modelado conceptual y generación de artefactos

La separación entre modelo conceptual y formato de serialización conduce a la necesidad de utilizar técnicas de modelado que permitan describir el dominio de forma independiente de una tecnología concreta. En este sentido, Unified Modeling Language (UML) constituye una referencia ampliamente utilizada para especificar, visualizar y documentar sistemas mediante clases, atributos, relaciones, jerarquías y multiplicidades [Object Management Group, 2026b]. Su utilidad en este trabajo reside en la posibilidad de representar conceptos curriculares y sus relaciones sin depender inicialmente de XML, JSON u otro formato de intercambio.

Por este motivo, UML se utilizará como notación principal para representar el esquema conceptual derivado del análisis del dominio. En particular, los diagramas de clases permiten expresar entidades curriculares, atributos, asociaciones y multiplicidades en un nivel intermedio entre la descripción normativa de CVN y los artefactos técnicos generados posteriormente. Esta representación no sustituye a los modelos ejecutables ni a los esquemas de validación, pero proporciona una vista comprensible y tecnológicamente independiente del esquema que se desea obtener.

Cuando el modelo requiere restricciones más precisas que las expresables mediante clases

y relaciones, Object Constraint Language (OCL) permite definir condiciones formales sobre modelos UML [Object Management Group, 2026a]. Estas restricciones pueden utilizarse para especificar reglas de validez, cardinalidad o consistencia que no siempre quedan reflejadas de forma natural en un diagrama estructural. Aunque este trabajo no se basa exclusivamente en UML/OCL como fuente de generación final, estos conceptos resultan relevantes para justificar la separación entre estructura, semántica y validación.

En esta línea, existen herramientas que permiten generar artefactos técnicos a partir de modelos. BESSER, por ejemplo, proporciona un generador de modelos Pydantic a partir de especificaciones de modelado [BESSER, 2026]. Este tipo de enfoque muestra una tendencia importante: definir el dominio en un nivel conceptual o declarativo y obtener después clases, validadores o esquemas de forma automática. La arquitectura de Open CVN adopta esta idea general de generación y trazabilidad, aunque la adapta a las fuentes disponibles en el ecosistema CVN y a las necesidades concretas del proyecto.

Otra herramienta relacionada es LinkML, un lenguaje de modelado orientado a datos estructurados que permite definir esquemas de forma declarativa y generar artefactos en distintos formatos [LinkML, 2026]. LinkML resulta especialmente interesante por su capacidad para actuar como puente entre modelos, esquemas JSON, documentación y tecnologías de datos enlazados. No obstante, para el alcance de este TFG, su adopción completa introduciría una capa adicional de complejidad que no resulta imprescindible. Por ello, se considera como referencia conceptual, pero no como dependencia central de la solución desarrollada.

2.5. Síntesis del estado del arte

El análisis realizado permite extraer varias conclusiones relevantes para el desarrollo de una representación curricular abierta. En primer lugar, los modelos relacionados con información de investigación, como CERIF, VIVO o ROH, muestran soluciones expresivas y semánticamente ricas, aunque su complejidad puede resultar desproporcionada para una herramienta abierta y ligera de gestión curricular. En segundo lugar, los formatos de serialización deben entenderse como mecanismos de intercambio y almacenamiento, no como sustitutos del modelo conceptual. En tercer lugar, UML ofrece una notación adecuada para representar el esquema conceptual de forma comprensible antes de materializarlo en modelos ejecutables o esquemas de validación.

Estas observaciones justifican la necesidad de una arquitectura propia que separe modelo conceptual, serialización y validación, preserve trazabilidad hacia las fuentes oficiales del dominio y genere una representación final más adecuada para validación, almacenamiento y

exportación. De este modo, el estado del arte no conduce a adoptar sin modificaciones una ontología completa ni a reproducir directamente una estructura de intercambio existente, sino a proponer una solución intermedia: suficientemente formal para ser validable, suficientemente trazable para conservar el significado curricular y suficientemente ligera para servir como base de herramientas abiertas.

3. Análisis del ecosistema CVN y propuesta de solución

El Capítulo 2 concluyó que una representación curricular abierta debe combinar una separación clara entre modelo conceptual, serialización y validación con una trazabilidad explícita hacia las fuentes normativas del dominio. El presente capítulo aplica esa conclusión al caso concreto de este Trabajo Fin de Grado. En primer lugar, se examina en detalle el contenido real del paquete oficial de CVN empleado como fuente del proyecto. A continuación, se identifican las limitaciones técnicas que dicho paquete introduce cuando se pretende utilizarlo directamente como modelo interno de una aplicación. Finalmente, a partir de ese análisis, se presenta la arquitectura Open CVN que se desarrolla con más detalle en los capítulos siguientes.

3.1. El paquete oficial CVN como fuente del proyecto

El Capítulo 1 ya situó a CVN como la norma española de referencia para la presentación de información curricular de investigadores, mantenida por FECYT [FECYT, 2026]. Ese capítulo, sin embargo, se limitó a introducir CVN como motivación del proyecto, sin describir el contenido técnico del paquete que FECYT distribuye para su implementación. Esa descripción es el punto de partida de este apartado.

El paquete oficial empleado como fuente de este trabajo no es un único fichero, sino un conjunto heterogéneo de artefactos que puede organizarse en cuatro capas complementarias. La primera capa es un manual de especificaciones técnicas en formato PDF, orientado a lectura humana, que define el significado funcional de cada campo curricular, su tipo de dato, su obligatoriedad y la tabla de referencia asociada cuando corresponde. La segunda capa traduce

ese mismo manual a un fichero XML estructurado, pensado para su explotación automática en lugar de su lectura directa. La tercera capa es un modelo en árbol que enlaza cada código funcional del manual con las propiedades e indicadores concretos que aparecen en un documento CVN real serializado en XML. La cuarta capa está formada por los esquemas XSD que definen la forma legal de ese XML, incluyendo un vocabulario extenso de tablas auxiliares codificadas como enumeraciones.

Junto a estas cuatro capas, el paquete incluye tres familias auxiliares que no forman parte del núcleo mínimo de codificación, pero que resultan necesarias para interpretar correctamente numerosas referencias del manual: un catálogo normalizado de entidades e instituciones, una materialización en XML de buena parte de las tablas del anexo de valores controlados junto con la codificación de subtipos, y un tesauro jerárquico multilingüe de materias y palabras clave. Sin estas familias auxiliares, una parte relevante de los campos del manual quedaría referida a catálogos externos sin forma de resolverse dentro del propio paquete.

La Tabla 3.1 resume, para cada una de estas fuentes, la información principal que aporta y el uso concreto que recibe dentro del proyecto.

Estas fuentes no son intercambiables entre sí. El manual en PDF y su versión estructurada en XML son la mejor base para entender *qué significa* cada campo, mientras que el modelo en árbol y los esquemas XSD explican *cómo se serializa* ese mismo campo dentro de un documento XML concreto. Esta distinción, introducida aquí de forma descriptiva, es la que justifica más adelante por qué la propuesta Open CVN separa explícitamente el análisis semántico del dominio de su representación técnica en XML.

Fuente	Información que aporta	Uso dentro del proyecto
Manual de especificaciones técnicas (PDF)	Semántica funcional de cada campo curricular, tipo de dato y obligatoriedad	Referencia normativa para interpretar el significado de los códigos CVN
Manual estructurado en XML	Catálogo del manual en forma parseable: código, nombre, tipo, obligatoriedad y tabla de referencia	Fuente principal para la extracción automática de metadatos funcionales
Modelo en árbol XML	Enlace entre cada código funcional y las propiedades e indicadores del XML CVN real	Trazabilidad estructural entre el dominio funcional y la serialización XML
Esquemas XSD del núcleo (documento, tipos comunes y tablas auxiliares)	Forma legal del XML final y vocabulario de tablas codificadas como enumeraciones	Generación reproducible de bindings estructurales y validación de forma
Tablas de referencia y subtipos	Materialización en XML de gran parte del anexo de valores controlados y de la codificación de subtipos	Resolución de referencias auxiliares y evaluación de enumeraciones cerradas
Catálogo de entidades	Registro normalizado de instituciones y entidades	Resolución de referencias de tipo registro
Tesauro multilingüe	Vocabulario jerárquico de materias y palabras clave	Resolución de referencias de tipo tesauro

Tabla 3.1: Fuentes del paquete oficial CVN analizadas y su uso en el proyecto.

3.2. El XML embebido en los documentos CVN

Además del paquete de especificación descrito en el apartado anterior, la herramienta oficial de FECYT genera, para cada currículo, un documento en formato PDF pensado principalmente para su presentación y lectura humana. Una parte relevante de estos documentos conserva, embebido dentro del propio fichero PDF, el XML CVN que dio lugar a su generación. Cuando ese XML embebido existe y resulta accesible, constituye una fuente adicional de información curricular ya estructurada, en lugar de un simple documento visual del que haya que extraer texto de forma no estructurada.

Esta circunstancia tiene una implicación práctica directa para el procesamiento automático de currículos ya existentes: un sistema que pretenda incorporar currículos previamente generados por la herramienta oficial puede intentar, en primer lugar, recuperar y validar ese XML embebido, y solo recurrir a mecanismos menos deterministas cuando dicha extracción no sea posible. Este trabajo adopta precisamente ese criterio de prioridad, cuyo desarrollo funcional completo se presenta en el Capítulo 6.

3.3. Limitaciones y dificultades técnicas detectadas

El análisis detallado del paquete oficial revela un conjunto de limitaciones técnicas que, sin restar validez a la norma CVN, condicionan de forma directa cualquier intento de utilizar sus artefactos como modelo interno de una aplicación. Estas limitaciones se agrupan en cuatro tipos.

En primer lugar, existen discrepancias puntuales entre el XML canónico del modelo en árbol y el esquema XSD que se supone que lo valida. El caso más representativo aparece en dos indicadores concretos del modelo en árbol, que incluyen un elemento hijo no declarado por dicho esquema. De los cientos de indicadores del mismo tipo presentes en el documento, únicamente dos contienen ese elemento adicional, y ambos hacen referencia al mismo valor de una tabla auxiliar de tipos de calidad. Esta discrepancia impide que unas estructuras generadas directamente desde el esquema puedan analizar por completo el XML canónico, y obliga a tratar dicho XML como evidencia de origen que debe registrarse explícitamente en lugar de forzarse a encajar en la forma legal definida por el esquema.

En segundo lugar, ciertas construcciones del propio lenguaje de esquemas no se trasladan de forma perfecta a estructuras de datos ejecutables. El mecanismo de composición *choice*, que en XSD permite declarar alternativas entre varios elementos posibles, no siempre puede imponerse como una restricción de exclusión mutua real sobre las estructuras generadas

automáticamente a partir del esquema. De forma similar, algunas cardinalidades mínimas declaradas en el esquema no quedan garantizadas por dichas estructuras, y varios atributos se generan con un tipo demasiado genérico, sin estructura interna definida. Estas limitaciones son consistentes con una idea ya avanzada en el Capítulo anterior: un esquema de validación describe la forma legal de un documento, pero no captura por completo las reglas semánticas del dominio que representa.

En tercer lugar, no todas las tablas auxiliares del paquete se comportan como enumeraciones cerradas. Algunas incluyen valores de tipo *delegado* o comportamiento abierto que impide tratarlas como un catálogo fijo sin perder información, mientras que otras sí son suficientemente estables y compactas para ese tratamiento. Determinar esta distinción exige examinar la evidencia concreta de cada tabla en lugar de asumir un criterio uniforme para todas ellas.

En cuarto lugar, algunas referencias mencionadas en el manual funcional no tienen una tabla equivalente clara dentro del resto del paquete, y las tres familias auxiliares descritas en el apartado anterior presentan una deriva histórica de empaquetado, con nombres de fichero que no siempre coinciden entre la documentación de orientación y los artefactos realmente distribuidos. Esta circunstancia obliga a resolver dichas familias mediante un mapeo consciente del paquete, en lugar de asumir convenios de nombre fijos.

En conjunto, estas cuatro limitaciones evidencian un mismo problema de fondo: la estructura XML del paquete oficial y el significado curricular que dicha estructura pretende representar no están completamente alineados entre sí. La Tabla 3.2 resume estas limitaciones junto con la respuesta de diseño que se adopta para cada una de ellas dentro de la arquitectura Open CVN.

Ninguna de estas limitaciones invalida la norma CVN ni el paquete distribuido por FECYT, que cumplen adecuadamente su función original de interoperabilidad documental. Lo que estas limitaciones muestran es que dicho paquete no puede tomarse, sin mediación adicional, como el modelo interno definitivo de una herramienta de gestión curricular. Esa mediación adicional es precisamente el objeto de la propuesta de solución que se presenta a continuación.

Limitación detectada	Respuesta de diseño
Discrepancias puntuales entre el XML canónico y su esquema de validación	Tratar el XML como evidencia de origen y registrar la discrepancia de forma explícita, sin forzar un análisis silencioso ni descartar el documento
Construcciones XSD que no se trasladan de forma perfecta a estructuras ejecutables (*choice* sin exclusión mutua, cardinalidades mínimas no garantizadas, atributos demasiado genéricos)	Confinar estas limitaciones a la capa estructural de bindings y no exponerlas como el contrato público de Open CVN
Tablas auxiliares que no siempre son enumeraciones cerradas	Evaluar la elegibilidad de enumeración cerrada a partir de evidencia concreta de cada tabla, en lugar de un criterio uniforme asumido de antemano
Referencias del manual sin tabla equivalente clara, y deriva de empaquetado entre familias auxiliares	Conservar estas referencias como no resueltas o parcialmente trazables en lugar de forzar una conversión con pérdida de información
Separación insuficiente entre estructura XML y significado curricular	Introducir capas intermedias explícitas de normalización, política semántica y modelo conceptual antes de definir el formato final

Tabla 3.2: Limitaciones detectadas en el paquete oficial CVN y respuesta de diseño de Open CVN.

3.4. Propuesta de solución: la arquitectura Open CVN

Copiar directamente la estructura del XML oficial como modelo interno de una aplicación resolvería la interoperabilidad con CVN, pero trasladaría a dicha aplicación todas las limitaciones descritas en el apartado anterior, y mezclaría dos preocupaciones de naturaleza distinta: la interoperabilidad con un formato de intercambio externo y el modelado semántico del dominio curricular. Por este motivo, Open CVN no adopta el XML oficial como modelo final, sino que lo trata como una fuente formal de la que se parte mediante un proceso reproducible y explícitamente trazable.

Esta decisión no implica renunciar a la relación con CVN. Al contrario: la propuesta exige conservar, para cada elemento del modelo final, evidencia verificable de la fuente CVN de la que procede, de modo que la transformación hacia un formato propio no suponga una pérdida de significado curricular. Esta exigencia de trazabilidad se traduce en una arquitectura organizada por capas, en la que cada capa añade una interpretación adicional sobre la capa anterior sin descartar la relación con el paquete oficial. La Figura 3.1 representa esta idea de forma esquemática.

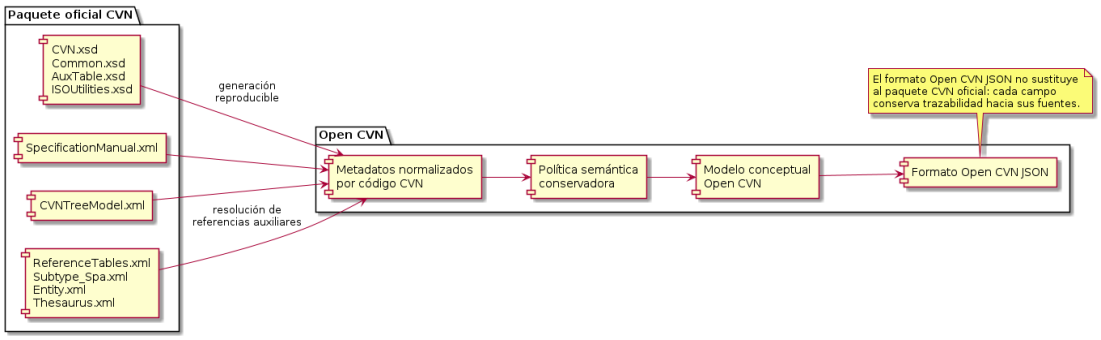


Figura 3.1: Separación entre el paquete oficial CVN y la arquitectura Open CVN.

Como muestra la Figura 3.1, el paquete oficial completo, incluidas sus familias auxiliares, se convierte en primer lugar en metadatos normalizados por código CVN, un paso que unifica en una única estructura la información dispersa entre el manual, el modelo en árbol y las fuentes auxiliares. Sobre esos metadatos normalizados actúa una política semántica conservadora, que decide de forma explícita y trazable cómo debe interpretarse cada campo: como texto, fecha, número, enumeración cerrada, catálogo abierto, referencia a una entidad, término de tesauro o referencia no resuelta, entre otros casos. El resultado de esta política alimenta un

modelo conceptual de dominio que ya no depende de la estructura concreta del XML CVN, y que constituye la base a partir de la cual se deriva finalmente el formato Open CVN JSON descrito en el Capítulo 6.

Este modelo conceptual se representa mediante diagramas de clases UML, tal y como se anticipó en el Capítulo 2. La Figura 3.2 muestra una vista compacta de dicho modelo, en la que un documento Open CVN se organiza en metadatos, un bloque de currículo y un espacio de extensión, y en la que ese bloque de currículo se subdivide en áreas conceptuales del dominio académico e investigador. Los nombres de clase de esta vista coinciden deliberadamente con las claves técnicas del formato Open CVN JSON, de modo que la figura sirva también como puente directo hacia la estructura raíz que se detalla en el Capítulo 6.

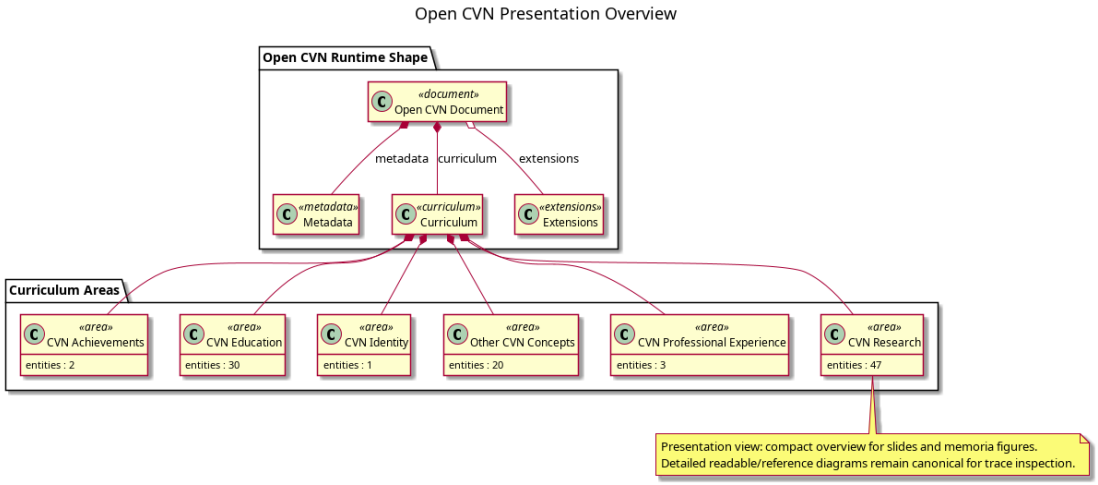


Figura 3.2: Vista UML compacta del esquema conceptual de Open CVN, con el número de conceptos identificados por área curricular.

Sobre esta base conceptual y de trazabilidad se construyen finalmente los mecanismos que hacen operativa la propuesta: un formato Open CVN JSON validable y extensible, y un conjunto de utilidades de análisis, validación e importación que, junto con una herramienta local de gestión, demuestran que la representación definida puede emplearse en flujos reales de trabajo curricular. Esta herramienta local cumple una función demostrativa del enfoque, mientras que la aportación principal de la propuesta reside en la arquitectura de representación, normalización y trazabilidad descrita en este apartado.

3.5. Alcance y exclusiones

El análisis anterior permite delimitar con precisión qué garantías ofrece realmente la arquitectura Open CVN frente al ecosistema CVN del que parte.

Se automatizan con garantías reproducibles los siguientes elementos:

- La generación de la capa estructural desde los esquemas oficiales.
- La normalización de metadatos por código CVN a partir del manual y del modelo en árbol.
- La resolución de referencias auxiliares cuando existe evidencia suficiente en el paquete.
- La aplicación de la política semántica conservadora.
- La generación del modelo conceptual y del formato Open CVN JSON a partir de dicho modelo.

Se mantienen como mapeo parcial, y así deben entenderse, los siguientes elementos:

- La interpretación de tablas auxiliares sin evidencia suficiente para una enumeración cerrada, que permanecen documentadas como abiertas o pendientes de revisión.
- La incorporación de currículos CVN ya existentes en formato XML, que se resuelve mediante un mapeo semántico de los elementos reconocidos del documento en lugar de una conversión completa y garantizada de cualquier XML CVN posible.

Quedan documentadas explícitamente como limitaciones o trabajo futuro:

- Las referencias del manual sin tabla equivalente clara en el resto del paquete.
- La deriva de empaquetado entre familias auxiliares.
- La ausencia de una garantía formal de que el modelo conceptual capture la totalidad de las reglas de negocio implícitas en el manual funcional.

Ninguna de estas exclusiones se oculta: todas ellas se retoman de forma explícita en la discusión de garantías del Capítulo 7.

En conjunto, el análisis realizado en este capítulo justifica por qué Open CVN no se limita a reproducir la estructura XML de CVN, sino que introduce una arquitectura propia, trazable y validable a partir de ella. El Capítulo siguiente describe la metodología de trabajo seguida y detalla la arquitectura general por capas que materializa esta propuesta.

4. Metodología, herramientas y arquitectura general

El Capítulo 3 analizó el contenido real del paquete oficial CVN, identificó sus limitaciones técnicas y presentó, a un nivel todavía conceptual, la arquitectura por capas que Open CVN propone frente a esas limitaciones. El presente capítulo desciende un nivel de concreción: describe el proceso de trabajo seguido durante el desarrollo, justifica la selección de herramientas empleadas y detalla la arquitectura general del sistema, incluida la organización del repositorio que la materializa. Los detalles de implementación de cada capa se reservan para el Capítulo 5.

4.1. Metodología de trabajo

El desarrollo de este Trabajo Fin de Grado no siguió un proceso de ingeniería del software orientado a requisitos de usuario o iteraciones de producto, sino un proceso adaptado a un trabajo de Computación centrado en el análisis de formatos, la generación reproducible de artefactos y la verificación técnica de un sistema. Este proceso puede describirse en cinco fases sucesivas.

La primera fase consistió en un análisis inicial del dominio curricular y de los formatos de representación disponibles, cuyo resultado se recoge en el Capítulo 2. La segunda fase exploró en detalle los artefactos oficiales del paquete CVN, identificando su estructura y sus limitaciones técnicas, tal y como se presentó en el Capítulo 3. La tercera fase, desarrollada en el presente capítulo, diseñó de forma incremental una arquitectura por capas capaz de absorber esas limitaciones sin perder trazabilidad hacia las fuentes oficiales. La cuarta fase implementó dicha arquitectura de forma reproducible, generando de manera automática los

artefactos estructurales, los metadatos normalizados, los modelos de dominio y el formato Open CVN JSON, como se describe en el Capítulo 5. La quinta fase verificó el sistema mediante pruebas automatizadas y flujos completos de extremo a extremo, cuyos resultados se discuten en el Capítulo 7.

Estas cinco fases no se ejecutaron de forma estrictamente secuencial. Cada limitación detectada durante la exploración del paquete oficial obligó, en la práctica, a revisar decisiones de diseño ya tomadas en fases anteriores, y cada nueva capa incorporada exigió documentar explícitamente las decisiones y las limitaciones que introducía. Esta documentación continua de decisiones y limitaciones técnicas, más que una fase aislada, constituye un criterio transversal de todo el proceso: ninguna limitación detectada se resolvió ocultándola, sino registrándola y decidiendo de forma explícita cómo debía tratarse en la capa correspondiente.

4.2. Herramientas empleadas

La selección de herramientas responde a un criterio común: cada herramienta debía permitir generar o validar artefactos de forma reproducible a partir de fuentes formales, sin introducir dependencias que condicionaran en exceso la portabilidad del sistema.

El proyecto se implementa en Python, cuya combinación de tipado gradual y ecosistema maduro de procesamiento de XML y JSON resulta adecuada para implementar las sucesivas capas de generación y transformación descritas en este documento. La gestión del entorno y de las dependencias se realiza con *uv*, un gestor de paquetes y proyectos Python que permite fijar de forma determinista tanto la versión del intérprete como las dependencias empleadas [Astral, 2026], un requisito necesario para que la generación de artefactos sea reproducible entre distintos entornos de ejecución.

La generación de la capa estructural a partir de los esquemas XSD oficiales se realiza con *xsdata*, una biblioteca de generación de enlaces de datos que permite obtener clases Python tipadas directamente desde esquemas XML sin necesidad de traducirlos manualmente [xsdata, 2026]. Esta herramienta se complementa con su extensión para Pydantic, que emite las clases generadas ya como modelos Pydantic en lugar de estructuras de datos genéricas.

Pydantic actúa como base de los modelos de dominio, del formato Open CVN JSON y de buena parte de la validación en tiempo de ejecución del sistema. Como ya se introdujo en el Capítulo 2, Pydantic combina anotaciones de tipo, validación de datos y generación de JSON Schema en un mismo mecanismo [Pydantic, 2026], lo que permite mantener alineados el código Python, el contrato de validación y la documentación estructural del formato. Ese contrato de validación se expresa como JSON Schema, cuya versión Draft 2020-12 es la

empleada para validar de forma declarativa el formato Open CVN JSON antes de aplicar las comprobaciones adicionales en tiempo de ejecución [JSON Schema, 2026].

El almacenamiento local de currículos, versiones y diagnósticos se apoya en SQLite, un motor de bases de datos embebido que no requiere un servidor independiente [SQLite, 2026]. Esta elección evita introducir una dependencia de infraestructura externa y mantiene la herramienta local autocontenida. La exportación a documentos LaTeX se implementa con Jinja, un motor de plantillas que separa la lógica de presentación de los datos ya validados [Pallets Projects, 2026], de modo que la generación de un documento LaTeX no exige mezclar código Python con marcado de presentación.

La verificación automatizada del sistema se realiza con *pytest* como marco de pruebas, junto con su extensión *pytest-xdist* para la ejecución en paralelo de la batería de pruebas [pytest, 2026], lo que permite mantener tiempos de ejecución razonables a pesar del volumen de comprobaciones necesarias para cubrir las distintas capas del sistema. El control de versiones se realiza con Git, y cada contribución se verifica de forma automática mediante GitHub Actions, un servicio de integración continua que ejecuta la batería de pruebas ante cualquier cambio propuesto sobre el repositorio [GitHub, 2026].

La Tabla 4.1 resume estas herramientas junto con su papel principal y la justificación de su uso dentro del proyecto.

Herramienta	Papel en el proyecto	Justificación
Python + *uv*	Lenguaje de implementación y gestión reproducible del entorno y las dependencias	Tipado gradual, ecosistema maduro para XML/JSON y fijación determinista del entorno de ejecución
xsdata + extensión Pydantic	Generación de bindings estructurales desde los esquemas XSD oficiales	Evita traducir manualmente esquemas complejos y mantiene los bindings alineados con la fuente oficial
Pydantic	Modelos de dominio, formato Open CVN JSON y validación en tiempo de ejecución	Combina tipado, validación y generación de JSON Schema en un mismo mecanismo
JSON Schema (Draft 2020-12)	Validación estructural declarativa del formato Open CVN JSON	Contrato de validación independiente del lenguaje de implementación
SQLite	Almacenamiento local de currículos, versiones y diagnósticos	Motor embebido que evita una dependencia de infraestructura externa
Jinja + motor LaTeX	Exportación de currículos almacenados a documentos LaTeX y PDF	Separa la plantilla de presentación de los datos ya validados
pytest + *pytest-xdist*	Verificación automatizada y ejecución paralela de la batería de pruebas	Mantiene tiempos de ejecución razonables sobre una batería de pruebas extensa
Git + GitHub Actions	Control de versiones e integración continua	Verifica automáticamente cada contribución antes de integrarse

Tabla 4.1: Herramientas empleadas en el proyecto y justificación de su uso.

4.3. Arquitectura general por capas

La arquitectura general del sistema traduce en artefactos concretos la separación conceptual presentada en el Capítulo 3 entre el paquete oficial CVN y el modelo Open CVN derivado de él. La Figura 4.1 representa esta arquitectura agrupada en cuatro grandes bloques, cada uno de los cuales corresponde a una parte concreta del repositorio del proyecto.

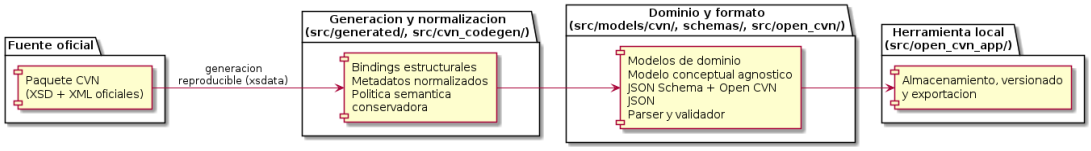


Figura 4.1: Arquitectura general de Open CVN organizada por capas, con el módulo del repositorio responsable de cada bloque.

Cada uno de estos cuatro bloques agrupa, a su vez, una secuencia de etapas más concretas. Los apartados siguientes describen esa secuencia bloque a bloque, indicando para cada etapa qué recibe, qué produce y qué módulo del repositorio la implementa. El detalle interno de cómo se implementa cada etapa, incluidos los algoritmos y los resultados numéricos de su ejecución, se reserva para el Capítulo 5; este apartado se limita a establecer el mapa general de responsabilidades entre etapas.

4.3.1. Fuente oficial CVN

El primer bloque no realiza ningún procesamiento propio: corresponde al paquete oficial CVN ya descrito en el Capítulo 3, y actúa como frontera externa y única fuente formal de la que parte el resto de la arquitectura. Ninguna etapa posterior incorpora información curricular que no proceda, directa o indirectamente, de este bloque.

4.3.2. Generación estructural y normalización

El segundo bloque transforma el paquete oficial en metadatos internos normalizados y semánticamente interpretados, en cuatro etapas sucesivas:

- **Generación estructural.** A partir de los esquemas XSD oficiales se generan de forma reproducible bindings Pydantic que reflejan fielmente la estructura del XML de CVN.

Esta etapa produce los bindings estructurales almacenados en `src/generated/` y no incorpora todavía ninguna interpretación semántica de los campos.

- **Normalización.** A partir del manual funcional en XML y del modelo en árbol, esta etapa unifica en una única estructura, indexada por código CVN, la información hasta entonces dispersa entre ambas fuentes. Recibe el paquete oficial y produce metadatos normalizados por código.
- **Resolución de referencias auxiliares.** Sobre los metadatos normalizados, esta etapa resuelve las referencias del manual hacia las familias auxiliares del paquete descritas en el Capítulo 3, añadiendo a cada metadato la evidencia de resolución disponible.
- **Política semántica.** A partir de los metadatos ya resueltos, esta etapa decide de forma explícita y trazable la forma semántica de cada campo (texto, fecha, número, enumeración cerrada, catálogo abierto, referencia a entidad, término de tesauro o referencia no resuelta, entre otras), produciendo una política semántica completa del dominio.

Las cuatro etapas de este bloque se implementan como lógica mantenida a mano en `src/cvn_codegen/`, con la única excepción de los bindings generados por la primera etapa, que se almacenan de forma separada en `src/generated/`.

4.3.3. Dominio, modelo conceptual y formato

El tercer bloque construye, a partir de la política semántica, las representaciones que ya no dependen de la estructura concreta del XML de CVN:

- **Modelos de dominio.** A partir de la política semántica se generan modelos de dominio Pydantic, almacenados en `src/models/cvn/` junto con un conjunto reducido de componentes de dominio compartidos mantenidos a mano.
- **Modelo conceptual agnóstico.** A partir de los modelos de dominio y de las decisiones de normalización y política semántica que los originaron, esta etapa construye el inventario conceptual detallado en el Capítulo 5, ya independiente de la serialización XML.
- **JSON Schema y formato Open CVN JSON.** A partir del modelo conceptual se genera el artefacto JSON Schema, almacenado en `schemas/`, que define de forma declarativa la estructura válida del formato Open CVN JSON.
- **Parser y validador.** Sobre el JSON Schema y los modelos de dominio se apoya el contrato público de análisis y validación, almacenado en `src/open_cvn/`, que expone

la importación de Open CVN JSON, CVN XML y CVN PDF junto con la validación en tiempo de ejecución.

Las tres primeras etapas de este bloque se mantienen en `src/cvn_codegen/` y `src/models/cvn/`; la última corresponde ya al módulo `src/open_cvn/`, que constituye el límite público del sistema hacia cualquier código externo que necesite leer o validar currículos.

4.3.4. Herramienta local

El cuarto bloque no añade una etapa de transformación de datos, sino que consume el contrato público de análisis y validación para ofrecer un uso operativo completo: inicialización de almacenamiento, importación y exportación, gestión de un currículo maestro y de versiones derivadas, selección de secciones y entradas, y exportación a LaTeX y PDF. Todo este bloque se implementa en `src/open_cvn_app/`.

La Tabla 4.2 consolida, para cada módulo del repositorio mencionado en los apartados anteriores, la responsabilidad que le corresponde dentro de la arquitectura Open CVN.

Módulo	Responsabilidad
`src/generated/`	Bindings estructurales Pydantic generados automáticamente desde los esquemas XSD oficiales; no se edita manualmente
`src/cvn_codegen/`	Lógica mantenida a mano: normalización, resolución de referencias auxiliares, política semántica, generación de modelos de dominio, extracción del modelo conceptual y generación de diagramas y JSON Schema
`src/models/cvn/`	Componentes de dominio compartidos mantenidos a mano y modelos de dominio generados automáticamente a partir de la política semántica
`schemas/`	Artefacto JSON Schema generado del formato Open CVN
`src/open_cvn/`	Contrato público de análisis y validación: importación de Open CVN JSON, CVN XML y CVN PDF, y validación en tiempo de ejecución
`src/open_cvn_app/`	Herramienta local: interfaz de línea de órdenes, almacenamiento SQLite, versionado, edición de selección y exportación a LaTeX y PDF

Tabla 4.2: Módulos del repositorio y responsabilidades dentro de la arquitectura Open CVN.

4.4. Contratos entre capas y razonamiento arquitectónico

La separación entre módulos descrita en el apartado anterior no es únicamente organizativa: cada frontera entre módulos corresponde a un contrato explícito que determina qué información puede cruzar de una capa a otra. Generar un modelo de dominio final directamente desde los esquemas XSD oficiales, sin capas intermedias, mezclaría dos preocupaciones de naturaleza distinta: la interoperabilidad estructural con el XML de CVN y el modelado semántico del dominio curricular. Por ello, los bindings estructurales generados automáticamente se tratan como una capa de fidelidad hacia el XML oficial, y cualquier limpieza semántica o ergonómica se realiza en capas posteriores, en lugar de modificarse a mano sobre el código generado.

Esta decisión tiene una consecuencia práctica directa sobre la reproducibilidad del sistema: dado que la capa estructural puede regenerarse en cualquier momento a partir de los esquemas oficiales sin perder trabajo manual, las limitaciones que dicha capa presenta, ya descritas en el Capítulo 3, quedan documentadas en lugar de enmascararse mediante ediciones manuales frágiles. De forma análoga, los modelos de dominio y el modelo conceptual no vuelven a inspeccionar el XML o los esquemas XSD originales, sino que consumen exclusivamente las decisiones ya tomadas por la normalización y por la política semántica, evitando así que la semántica del dominio se derive de forma inconsistente en distintos puntos del sistema. El contrato público de análisis y validación, a su vez, expone únicamente el formato Open CVN JSON y sus resultados de validación, sin obligar a quien lo utiliza a conocer los detalles internos de generación de las capas anteriores.

En conjunto, esta arquitectura por capas materializa de forma concreta la propuesta presentada en el Capítulo 3: conserva trazabilidad hacia el paquete oficial CVN en cada etapa, mantiene separadas la interoperabilidad estructural y la semántica de dominio, y permite verificar de forma independiente cada capa mediante pruebas automatizadas. El Capítulo 5 describe en detalle cómo se implementa cada una de estas capas, desde la generación estructural hasta la extracción del modelo conceptual y la generación de diagramas y JSON Schema.

5. Implementación del pipeline de generación y normalización

El Capítulo 4 presentó la arquitectura general de Open CVN agrupada en cuatro bloques y detalló, dentro de cada uno, la secuencia de etapas que los componen. El presente capítulo desciende al último nivel de concreción antes de la evaluación del sistema: describe cómo se implementa cada una de esas etapas, qué recibe y qué produce, y qué evidencia concreta deja el sistema para trazar un campo desde el paquete oficial CVN hasta el formato Open CVN. El resultado funcional de este pipeline, el propio formato Open CVN JSON y la herramienta local que lo consume, se presentan en el Capítulo 6.

5.1. Generación estructural

La primera etapa del pipeline genera, de forma reproducible, los bindings Pydantic estructurales a partir de los esquemas XSD oficiales descritos en el Capítulo 3. Esta generación se ejecuta mediante *xsdata* a través de un componente de orquestación propio del repositorio, que valida los requisitos previos, limpia los directorios de salida existentes, ejecuta la generación para cada esquema y verifica que el resultado sea importable antes de darla por completada. La generación se invoca mediante un comando único que procesa de manera secuencial los siete paquetes estructurales del sistema: el documento CVN principal, el manual de especificaciones, el modelo en árbol, las tablas de referencia, los subtipos, el catálogo de entidades y el tesauro.

El modelo en árbol requiere, además, una configuración específica para evitar un fallo de generación por dependencias circulares, ya documentado como limitación en el Capítulo 3. Esta configuración se aplica únicamente a ese paquete, en lugar de alterar la configuración

general de generación para el resto de esquemas, de modo que la excepción quede acotada y explícita.

5.2. Normalización de metadatos por código CVN

Antes de aplicar cualquier interpretación semántica, el sistema necesita una representación única de cada campo curricular, ya que su información aparece dispersa entre el manual funcional y el modelo en árbol. La etapa de normalización resuelve esta dispersión: extrae de forma independiente los metadatos del manual en XML y las rutas técnicas del modelo en árbol, y a continuación los unifica en una única entrada por código CVN.

Sobre el paquete oficial descrito en el Capítulo 3, esta etapa produce actualmente **1457** entradas normalizadas. De ellas, **27** proceden únicamente del manual funcional y **1** únicamente del modelo en árbol; las **1429** restantes están presentes en ambas fuentes. El proceso registra, además, **33** discrepancias entre fuentes, entre las que se incluye la discrepancia del modelo en árbol ya analizada en el Capítulo 3.

El Código 5.1 muestra, de forma reducida y simplificada, el resultado normalizado del campo de código `000.010.000.030`, correspondiente al sexo de la persona, antes de que la política semántica decida su forma final.

```
codigo: "000.010.000.030"
campo_manual: "Sexo"
tabla_referencia_manual: "CVN_SEX_A"
ruta_arbol: "Identification/PersonalIdentification/Gender"
resolucion_referencia:
  estado: resuelto
  fuente: "ReferenceTables.xml"
  evidencia: 2 valores (Hombre, Mujer)
```

Código 5.1: Fragmento reducido del metadato normalizado del campo de sexo.

5.3. Resolución de referencias auxiliares

Una parte considerable de los campos del manual remite a una tabla de valores controlados en lugar de admitir texto libre. La etapa de resolución de referencias auxiliares comprueba,

para cada campo con una tabla asociada, si dicha tabla puede resolverse contra las tablas de referencia, los subtipos, el catálogo de entidades o el tesauro descritos en el Capítulo 3, y registra el resultado como evidencia explícita adjunta al metadato normalizado.

De las 1457 entradas normalizadas, **557** incluyen una tabla de referencia en el manual y, por tanto, pasan por esta etapa; las **900** restantes corresponden a campos de texto, fecha o número sin tabla asociada. La Tabla 5.1 desglosa esas 557 entradas según el tipo de evidencia de resolución obtenida.

Tipo de evidencia de resolución	**Entradas**
Tabla compacta de tipo enumeración	258
Clasificación temática jerárquica	125
Registro de entidad de un paquete auxiliar	93
Clasificación de ámbito o alcance	16
Tesauro o vocabulario de un paquete auxiliar	27
Familia controlada respaldada por subtipos	20
Tabla de tipo identificador	11
Escala o medida compacta	6
Referencia no resuelta	1
Total con tabla de referencia	**557**

Tabla 5.1: Resultado de la resolución de referencias auxiliares sobre las entradas normalizadas con tabla asociada.

Estos resultados confirman con evidencia concreta dos afirmaciones ya adelantadas en el Capítulo 3. Por un lado, únicamente una entrada permanece sin resolver dentro del paquete analizado: el campo de código `060.010.000.030` (*Fuente de índice H*), cuya tabla declarada en el manual no tiene equivalente directo en el resto del paquete. Por otro lado, la tabla del campo de sexo (`000.010.000.030`) se resuelve contra las tablas de referencia con solo dos valores posibles, sin jerarquía, delegados ni entradas duplicadas, lo que constituye la evidencia que permite tratarla como una enumeración cerrada en la etapa siguiente.

5.4. Política semántica

Los esquemas XSD oficiales solo indican cómo se serializa un campo, no cómo debe representarse en un modelo de dominio: un mismo tipo XSD genérico puede esconder tanto un texto libre como un conjunto cerrado de valores controlados, y una tabla auxiliar puede

ser tan pequeña como la del campo de sexo o tan abierta como un catálogo institucional en crecimiento. El Capítulo 3 ya identificó esta ambigüedad como una limitación del paquete oficial. La política semántica es la etapa que la resuelve: toma los metadatos normalizados y, cuando existe, la evidencia de resolución de la etapa anterior, y decide de forma explícita y trazable qué forma debe adoptar cada campo en el modelo de dominio, sin dejar nunca esa decisión implícita en el tipo XSD de origen.

5.4.1. Formas semánticas reconocidas

La política reconoce nueve formas semánticas base. La Tabla 5.2 las describe junto con un ejemplo real tomado del paquete oficial CVN.

Forma semántica	**Qué representa**	**Ejemplo**
Texto	Cadena libre sin tabla de valores asociada	Título de una publicación
Fecha	Valor temporal, completo o parcial	Fecha de inicio de un contrato
Número	Valor numérico	Número de horas impartidas
Enumeración cerrada	Tabla auxiliar con evidencia suficiente para tratarse como un conjunto fijo de valores	Sexo (tabla `CVN_SEX_A`)
Catálogo abierto	Tabla auxiliar cuya evidencia impide tratarla como cerrada	Tipo de entidad (tabla `CVN_ENTITY_TYPE`)
Referencia a entidad	Remite al catálogo de instituciones y organizaciones del paquete auxiliar	Entidad que participa en un proyecto
Término de tesauro	Remite al vocabulario jerárquico de materias del paquete auxiliar	Palabra clave de una publicación
Valor respaldado por subtipo	Remite a la codificación de subtipos derivada de una tabla auxiliar	Propiedad industrial (tabla `CVN_KNOW_A`)
Referencia no resuelta	La tabla declarada en el manual no tiene equivalente localizable en el resto del paquete	Fuente de índice H (tabla `CVN_AGENCY_C`)

Tabla 5.2: Formas semánticas reconocidas por la política semántica.

Las tres primeras formas se asignan directamente a los campos que no declaran ninguna tabla de referencia en el manual, es decir, a las 900 entradas identificadas en el apartado anterior. Las seis formas restantes se deciden a partir de la evidencia de resolución obtenida para las 557 entradas que sí declaran una tabla, y son las que exigen una regla de decisión

más cuidadosa.

5.4.2. Enumeración cerrada frente a catálogo abierto

La distinción más delicada de la política es la que separa una enumeración cerrada de un catálogo abierto, porque ambas parten del mismo punto: una tabla compacta de tipo enumeración, ya identificada en la Tabla 5.1. La política no decide esta distinción por el número de valores de la tabla, sino por su comportamiento: declara ineligible para enumeración cerrada cualquier tabla que presente jerarquía interna o un valor delegado, es decir, un valor que remite a un texto libre adicional en lugar de cerrar el conjunto de opciones.

Los dos ejemplos ya utilizados en este capítulo ilustran por qué el tamaño por sí solo no basta. La tabla del campo de sexo (`CVN_SEX_A`) contiene únicamente 2 valores, sin jerarquía ni delegados, y se declara elegible como enumeración cerrada. La tabla del campo de tipo de entidad (`CVN_ENTITY_TYPE`) contiene 17 valores, un tamaño que podría parecer igualmente manejable, pero incluye un valor delegado de tipo "Organismo, Otros" y una etiqueta repetida; por esa evidencia, la política la declara ineligible y la trata como catálogo abierto. Tratar `CVN_ENTITY_TYPE` como una enumeración cerrada habría obligado a elegir, para cualquier entidad que no encajase en las 17 categorías previstas, entre forzar un valor incorrecto o rechazar el dato; el catálogo abierto evita ambos problemas conservando el valor literal declarado.

5.4.3. Presencia, nomenclatura y traza

Además de la forma semántica, la política determina si un campo es obligatorio u opcional y si admite un único valor o varios, y propone un nombre de dominio en español derivado del nombre que el campo recibe en el manual: el campo manual "Sexo" se convierte, por ejemplo, en el atributo de dominio `sexo`, obligatorio y de valor único. Cada una de estas decisiones queda acompañada de una traza explícita que identifica la regla aplicada y su nivel de confianza, de modo que cualquier campo del modelo de dominio pueda justificarse frente a la fuente CVN concreta de la que procede, sin depender de inspeccionar de nuevo el XML o el XSD originales.

5.5. Generación de modelos de dominio y del modelo conceptual

La política semántica decide, campo a campo, cómo debe representarse cada dato; esta etapa aplica esa decisión dos veces, con dos propósitos distintos que conviene no confundir.

En primer lugar, el generador de modelos de dominio traduce la política semántica completa en clases Pydantic ejecutables: un artefacto de implementación, en Python, que ya no expone la estructura XML subyacente. La generación canónica produce actualmente **105** archivos: 101 módulos correspondientes a otros tantos conceptos curriculares del modelo en árbol, un módulo para los campos presentes únicamente en el manual, otro para los nodos del árbol sin código de manual asociado, y un módulo de enumeraciones con las **13** tablas que la política semántica clasificó como enumeración cerrada.

En segundo lugar, el extractor conceptual reagrupa esa misma información, sin volver a inspeccionar el XML o el XSD, en una representación ya independiente de Python: áreas curriculares, entidades conceptuales, atributos y vocabularios. Esta segunda representación es necesaria porque los modelos de dominio siguen atados a un lenguaje de implementación concreto, mientras que los diagramas UML y el formato Open CVN JSON necesitan una descripción agnóstica de la que partir. Siguiendo el ejemplo de este capítulo, el campo de sexo se integra así en la entidad conceptual `Person` del área `identity`, la misma agrupación en la que reaparece su definición final en la Sección 5.6.

5.6. Generación de JSON Schema y trazabilidad

La última etapa del pipeline genera, a partir del inventario conceptual, el artefacto JSON Schema que valida de forma declarativa el formato Open CVN JSON descrito en el Capítulo 6. El esquema generado declara la versión Draft 2020-12 y contiene actualmente **182** definiciones, de las cuales **74** corresponden a vocabularios controlados y el resto a entidades conceptuales y componentes de valor compartidos.

Cada definición generada conserva, junto a su forma de validación, un conjunto de anotaciones no validantes con el prefijo `x-open-cvn-`, que registran el código CVN de origen, la referencia de tabla utilizada, la elegibilidad de enumeración decidida y las rutas XML asociadas. Estas anotaciones son el mecanismo concreto de trazabilidad que conecta cada definición del formato Open CVN con la fuente CVN de la que procede. La Figura 5.1 recorre esta cadena completa para el campo de sexo ya utilizado como ejemplo en este capítulo, desde sus dos fuentes en el paquete oficial hasta su definición final en el esquema Open CVN.

Esta misma cadena de trazabilidad se aplica, sin excepción, a cualquier campo que atraviese el pipeline, incluidos aquellos que terminan en una referencia no resuelta: en esos casos, la anotación conserva la tabla original citada por el manual y un mensaje de diagnóstico explícito, en lugar de omitir el campo o de asignarle una forma semántica no justificada.

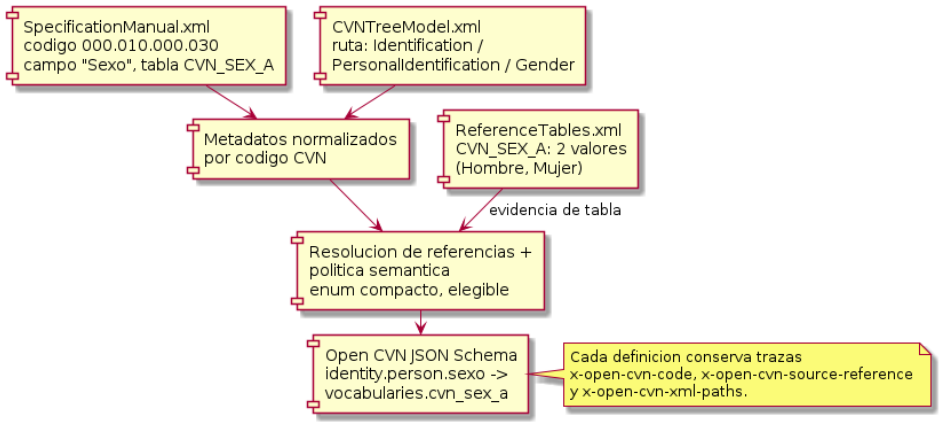


Figura 5.1: Trazabilidad de un campo desde el paquete oficial CVN hasta el esquema Open CVN JSON.

5.7. Síntesis de las etapas del pipeline

La Tabla 5.3 resume las etapas descritas en este capítulo, indicando para cada una su entrada, su salida y el módulo del repositorio que la implementa, ya presentado con mayor detalle arquitectónico en el Capítulo 4.

En conjunto, este pipeline demuestra que la trazabilidad hacia CVN exigida en el Capítulo 3 no es una declaración de intenciones, sino una propiedad verificable de cada campo generado. El Capítulo 6 retoma el resultado final de este pipeline, el formato Open CVN JSON, y describe su validación, su importación y la herramienta local que lo utiliza.

Etapa	Entrada	Salida	Módulo
Generación estructural	Esquemas XSD oficiales	Bindings Pydantic estructurales	`src/generated/`
Normalización	Manual funcional y modelo en árbol	Metadatos unificados por código CVN	`src/cvn_codegen/`
Resolución de referencias auxiliares	Metadatos normalizados y familias auxiliares	Evidencia de resolución por campo	`src/cvn_codegen/`
Política semántica	Metadatos normalizados y resueltos	Forma semántica trazable por campo	`src/cvn_codegen/`
Generación de modelos de dominio	Política semántica	Modelos de dominio Pydantic	`src/models/cvn/`
Extracción del modelo conceptual	Modelos de dominio y decisiones previas	Inventario conceptual agnóstico	`src/cvn_codegen/`
Generación de diagramas	Inventario conceptual	Diagramas UML en PlantUML	`src/cvn_codegen/`
Generación de JSON Schema	Inventario conceptual	Esquema Open CVN JSON	`schemas/`

Tabla 5.3: Etapas del pipeline de generación y normalización, entrada, salida y módulo responsable.

6. Formato Open CVN, validación y herramienta de gestión

El Capítulo 5 describió cómo el pipeline transforma el paquete oficial CVN en un modelo conceptual y en un artefacto JSON Schema trazable. El presente capítulo presenta el resultado funcional de ese trabajo: el formato Open CVN JSON que consumen las aplicaciones, el parser y el validador públicos que lo comprueban, los mecanismos de importación desde currículos ya existentes, y la herramienta local que demuestra el uso completo del sistema. Ninguno de estos elementos desplaza la aportación principal del trabajo, ya presentada en los capítulos anteriores: son su consecuencia operativa, no un objetivo en sí mismos.

6.1. El formato Open CVN JSON

Todo documento Open CVN es un objeto JSON con cuatro campos en su raíz:

- `schema_version`: identifica la versión del formato, no la del JSON Schema.
- `metadata`: describe el documento, no su contenido curricular.
- `curriculum`: contiene la información curricular propiamente dicha.
- `extensions`: opcional, reservado para metadatos específicos de una herramienta o proyecto concreto.

El campo `curriculum` organiza la información por área conceptual del dominio, en lugar de por estructura XML o por módulo generado: `identity` describe a la persona titular y es un único objeto, mientras que `education`, `research`, `professional_experience`, `achievements` y `other` son listas de entradas repetidas. Cada entrada de una lista declara

un `id` local opcional, un `type` conceptual estable, un objeto `data` con los campos propios de la entrada y, opcionalmente, un bloque `trace` con los códigos CVN y las rutas XML de los que procede.

Los campos que remiten a una tabla de valores controlados, ya introducidos en la Sección 5.4, comparten una misma forma en el JSON: un objeto con `code`, `label`, `source` y, cuando procede, `raw_value` o `uri`. El campo de sexo del ejemplo utilizado en el capítulo anterior se representa, siguiendo esta forma, como `{"code": "000", "label": "Mujer", "source": "CVN_SEX_A"}`. Esta misma forma se reutiliza tanto para las enumeraciones cerradas como para los catálogos abiertos: lo único que cambia entre ambos casos es si el validador puede restringir `code` a un conjunto cerrado de valores o debe aceptar cualquier valor declarado. El Código 6.1 recoge esta estructura raíz en su forma mínima, sin ningún dato curricular.

```
{
  "schema_version": "0.1.0",
  "metadata": {
    "language": "es",
    "policy": {"name": "default_cvn_semantic_policy", "version":
       "0.1.0"}
  },
  "curriculum": {
    "identity": {},
    "education": [],
    "research": [],
    "professional_experience": [],
    "achievements": [],
    "other": []
  }
}
```

Código 6.1: Estructura raíz mínima de un documento Open CVN JSON.

6.2. Parser y validador

El sistema expone un contrato público de análisis y validación con cuatro funciones: una para leer Open CVN JSON, otra para validarlo explícitamente, y dos más para importar currículos existentes en CVN XML y en CVN PDF. Las cuatro funciones devuelven una misma

estructura de resultado, con un estado de validación entre cinco valores posibles: sin ejecutar, válido, válido con avisos, inválido o fallido; una lista de errores y otra de avisos, cada uno con un código estable; y una traza que conserva el formato de origen, los códigos CVN detectados y las rutas XML asociadas.

Un documento se acepta con avisos, en lugar de aceptarse sin más, cuando supera la validación estructural y en tiempo de ejecución pero presenta indicios semánticos conservadores: por ejemplo, una entrada cuyo tipo conceptual no coincide con la sección en la que aparece, o una referencia controlada sin ningún campo de `code`, `label`, `raw_value` ni `uri`. Estos avisos nunca convierten un documento válido en inválido; se limitan a señalar un caso semánticamente atípico para que pueda revisarse.

La Figura 6.1 recorre el flujo completo desde la entrada, en cualquiera de los tres formatos admitidos, hasta la clasificación final del resultado.

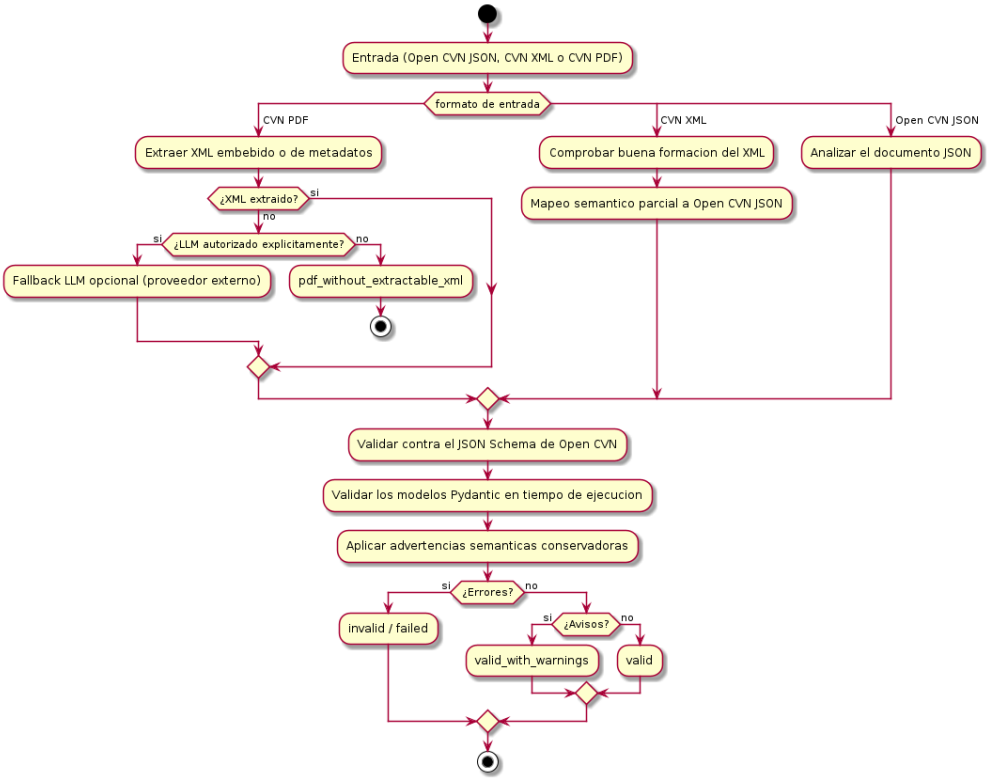


Figura 6.1: Flujo de importación y validación de currículos hacia Open CVN.

6.3. Importación de currículos existentes

La importación de Open CVN JSON ya validado es el caso directo: el documento se analiza y se valida contra el JSON Schema y contra los modelos de dominio, sin ninguna transformación adicional.

La importación desde CVN XML es semántica parcial: el documento se comprueba como XML bien formado y, a continuación, los elementos y campos reconocidos se traducen a secciones de Open CVN utilizando las mismas anotaciones de trazabilidad del JSON Schema descritas en el Capítulo 5. Los elementos no reconocidos no se descartan en silencio: quedan preservados en diagnósticos de importación y, cuando procede, en la sección `other` del currículo resultante. Esta importación no debe interpretarse como un conversor completo de cualquier CVN XML real, sino como un mapeo creciente sobre los casos ya cubiertos.

La importación desde CVN PDF prioriza siempre la vía determinista: si el PDF conserva el XML CVN embebido, o XML en sus metadatos, ese XML se extrae y sigue el mismo camino que la importación XML. Solo cuando esa extracción determinista no es posible, y únicamente si la persona usuaria lo autoriza de forma explícita, se recurre a un proveedor de modelos de lenguaje (LLM) como alternativa opcional. El resultado del proveedor externo se almacena únicamente después de validarse como Open CVN JSON, y queda marcado en las extensiones del documento como asistido por LLM, de modo que nunca se presenta como equivalente a una importación determinista.

6.4. Herramienta local de gestión

La herramienta local demuestra el uso completo del formato y de los mecanismos anteriores mediante una interfaz de línea de órdenes. Su modelo de datos distingue un currículo maestro, que representa la versión completa de referencia, de versiones derivadas que seleccionan secciones o entradas concretas de ese maestro para un propósito determinado, como una convocatoria o una solicitud específica. Toda la información se almacena en un fichero SQLite local de un único usuario.

El Código 6.2 y el Código 6.3 muestran, de forma reducida, un currículo maestro con una entrada de investigación y la versión derivada que excluye dicha entrada, tal y como quedaría tras ejecutar `versions exclude` sobre el maestro: el resto del currículo permanece intacto, y la versión derivada registra en sus extensiones qué selección se aplicó, lo que permite regenerarla de forma determinista a partir del maestro en lugar de mantenerla como una copia independiente que pueda divergir silenciosamente de su origen. La Tabla 6.1 resume, además,

los comandos principales de la herramienta.

Comando	Función
`store init`	Inicializa el almacenamiento SQLite local
`json import`	Importa un documento Open CVN JSON, opcionalmente como maestro
`pdf import`	Importa un CVN PDF, con extracción determinista y LLM opcional
`versions derive`	Crea una versión derivada a partir del maestro
`versions list`	Lista las versiones almacenadas
`versions sections` / `entries`	Explora las secciones y entradas de una versión
`versions include` / `exclude`	Selecciona o excluye una sección o entrada de una versión derivada
`versions metadata`	Asigna metadatos descriptivos a una versión derivada
`json export`	Exporta una versión almacenada como Open CVN JSON
`latex export`	Exporta una versión almacenada a LaTeX
`pdf generate`	Compila la exportación LaTeX a PDF cuando hay un motor TeX disponible

Tabla 6.1: Comandos principales de la herramienta local de gestión.

```
{
  "schema_version": "0.1.0",
  "curriculum": {
    "identity": {
      "sex": {
        "code": "000",
        "label": "Mujer",
        "source": "CVN_SEX_A"
      }
    },
    "research": [
      {
        "id": "research-001",
        "type": "research.publication",
        "data": {"title": "Open CVN data representation"}
      }
    ]
  }
}
```

Código 6.2: Currículo maestro reducido, con una entrada de investigación.

```
{
  "schema_version": "0.1.0",
  "curriculum": {
    "identity": {
      "sex": {
        "code": "000",
        "label": "Mujer",
        "source": "CVN_SEX_A"
      }
    },
    "research": []
  },
  "extensions": {
    "x-open-cvn.versioning": {
      "version_name": "publica",
      "version_kind": "derived",
      "selection": {
        "mode": "include_all",
        "excluded_pointers": ["/curriculum/research/0"]
      }
    }
  }
}
```

Código 6.3: Versión derivada “publica” del mismo currículo, con la entrada de investigación excluida.

6.5. Alcance funcional y garantías

No todos los flujos descritos en este capítulo ofrecen el mismo nivel de garantía, y conviene distinguirlos con precisión antes de la evaluación del sistema en el Capítulo 7. Los cuatro flujos principales se sitúan en puntos distintos entre una garantía determinista y una aproximación parcial y documentada:

- **Validación de Open CVN JSON.** La validación estructural contra el JSON Schema y la validación en tiempo de ejecución contra los modelos Pydantic son deterministas y reproducibles para cualquier documento Open CVN JSON: dado el mismo documento, el resultado de la validación es siempre el mismo, y no depende de heurísticas ni de servicios externos.
- **Importación desde CVN XML.** Es un mapeo semántico parcial, creciente sobre los casos ya cubiertos por el sistema. No debe presentarse ni interpretarse como una conversión completa de cualquier CVN XML real: los elementos no reconocidos se preservan como diagnóstico, pero no se traducen automáticamente al modelo Open CVN.
- **Importación asistida por LLM.** Es opcional y depende del proveedor externo configurado en cada caso. Su resultado nunca se acepta directamente: debe pasar primero por la misma validación de Open CVN JSON que cualquier otro documento, exactamente igual que los dos flujos anteriores.
- **Generación de PDF.** Depende de que exista un motor LaTeX disponible en el entorno de ejecución. Su ausencia no es un fallo del sistema ni impide el resto de los flujos: la importación, la validación, el almacenamiento y la exportación a JSON o LaTeX siguen funcionando sin necesidad de generar PDF.

El Capítulo 7 retoma estas cuatro distinciones para evaluar de forma sistemática qué partes del sistema ofrecen garantías fuertes y cuáles deben entenderse como una aproximación parcial y documentada.

7. Evaluación, resultados y discusión

Los capítulos anteriores han presentado la arquitectura Open CVN, su implementación por capas y el resultado funcional que ofrece a quien la utiliza. El presente capítulo cierra ese recorrido con una pregunta distinta: qué evidencia respalda, de forma reproducible, cada una de las afirmaciones técnicas realizadas hasta ahora. La evaluación no se limita a comprobar que el sistema se ejecuta sin errores, sino que verifica, capa a capa, que cada contrato descrito en los capítulos anteriores se cumple sobre el paquete oficial CVN y sobre los artefactos que de él se derivan.

7.1. Diseño de la evaluación

La evaluación se apoya en una batería de pruebas automatizadas organizada según las mismas capas de la arquitectura presentada en el Capítulo 4, en lugar de tratarse como un bloque monolítico. Cada nivel de evaluación comprueba una responsabilidad concreta:

- **Generación estructural:** produce artefactos íntegros e importables.
- **Normalización y resolución de referencias auxiliares:** mantienen la trazabilidad esperada.
- **Política semántica:** aplica sus reglas de forma consistente con la evidencia disponible.
- **JSON Schema generado:** valida correctamente documentos válidos e inválidos.
- **Parsers e importadores:** cumplen su contrato público.
- **Almacenamiento y versionado:** preservan la integridad de los datos.
- **Exportación:** produce artefactos deterministas.

- **Flujos completos de la herramienta:** funcionan de extremo a extremo.

Esta batería se ejecuta localmente durante el desarrollo y, además, de forma automática en cada contribución propuesta sobre las ramas principales del repositorio mediante integración continua, de modo que ninguna funcionalidad nueva se incorpora sin que el conjunto completo de garantías ya verificadas siga cumpliéndose.

7.2. Niveles de evaluación

La Tabla 7.1 resume los ocho niveles de evaluación considerados, el objetivo de cada uno y el número de pruebas automatizadas que lo cubren actualmente.

Nivel de evaluación	Objetivo	Pruebas
Artefactos generados	Bindings estructurales, modelos de dominio, modelo conceptual y diagramas íntegros e importables	146
Normalización y trazabilidad	Unificación manual/árbol por código CVN y resolución de referencias auxiliares correctas	90
Política semántica	Formas semánticas y elegibilidad de enumeración consistentes con la evidencia disponible	76
JSON Schema y validación runtime	Validación estructural y en tiempo de ejecución de documentos válidos e inválidos	25
Parsers e importadores	Contrato público de análisis, importación JSON/XML/PDF y casos de error	63
Almacenamiento y versiones	Persistencia SQLite, currículo maestro y versiones derivadas	25
Exportación	Generación determinista de LaTeX y PDF	19
Flujos de extremo a extremo	Pipeline completo y flujo completo de la herramienta local	44
Total		**488**

Tabla 7.1: Categorías de pruebas automatizadas y objetivo de cada una.

Esta organización por niveles permite además delimitar responsabilidades: una regresión detectada en el nivel de política semántica, por ejemplo, se diagnostica sin necesidad de examinar la herramienta local o los mecanismos de exportación, porque cada nivel verifica un contrato independiente de los demás.

7.3. Comando de verificación y resultados

La verificación completa del sistema se ejecuta con un único comando:

```
uv run pytest -n auto tests
```

Código 7.1: Comando principal de verificación del sistema.

Este comando ejecuta en paralelo las 488 pruebas de la Tabla 7.1 sobre el paquete oficial CVN vigente en el repositorio, sin datos simulados ni resultados precalculados. La ejecución más reciente sobre esta versión del documento superó las 488 pruebas sin ningún fallo, en 692,80 segundos (11 minutos y 32 segundos) de ejecución paralela. El mismo comando es el que ejecuta la integración continua ante cualquier contribución propuesta sobre las ramas principales del repositorio, por lo que el resultado no depende del entorno de quien lo ejecuta.

7.4. Discusión de garantías y limitaciones

Superar la batería de pruebas certifica que el sistema se comporta según lo descrito en los capítulos anteriores, pero no todas las partes del sistema ofrecen el mismo tipo de garantía. Conviene distinguir, en particular, las limitaciones que proceden del propio paquete oficial CVN de las que proceden de una decisión deliberada de alcance de este Trabajo Fin de Grado.

Las limitaciones que proceden del paquete oficial no son atribuibles al sistema desarrollado, sino a inconsistencias o a la propia naturaleza del material fuente:

- **Discrepancia XML/XSD del modelo en árbol.** El XML canónico y su esquema de validación no coinciden por completo, tal y como se documentó en el Capítulo 3.
- **Referencia `CVN_AGENCY_C` sin resolver.** El paquete no ofrece una tabla equivalente localizable para esta referencia del manual.
- **Deriva de empaquetado entre familias auxiliares.** Obliga a un mapeo consciente del paquete en lugar de un convenio de nombres fijo.

El sistema no oculta estas circunstancias: las registra explícitamente y las trata mediante evidencia en lugar de forzar una interpretación no justificada.

Las limitaciones que proceden del alcance del propio trabajo son, en cambio, decisiones deliberadas que delimitan hasta dónde llega la propuesta actual:

- **Importación de CVN XML.** Es un mapeo semántico parcial y creciente, no un conversor completo de cualquier documento CVN XML real.
- **JSON Schema generado.** No puede expresar toda regla semántica del dominio ni todas las relaciones conceptuales curadas.
- **Importación asistida por LLM.** Es opcional, y su resultado no se considera autoritativo hasta que se revisa.
- **Generación de PDF.** Depende de la disponibilidad de un motor LaTeX en el entorno de ejecución.

La Tabla 7.2 resume estas garantías junto con la limitación conocida que las acompaña.

Garantía verificada	Limitación conocida asociada
Generación estructural reproducible desde los esquemas XSD oficiales	Algunas construcciones XSD (*choice*, cardinalidades mínimas) no se preservan de forma estricta en los bindings generados
Normalización y resolución de referencias auxiliares con evidencia trazable	Una referencia del paquete analizado permanece sin resolver (`CVN_AGENCY_C`)
Política semántica evaluada a partir de evidencia concreta, no de listas fijas	Algunas tablas compactas quedan como revisión pendiente en lugar de enumeración cerrada o catálogo abierto definitivo
Validación estructural y en tiempo de ejecución determinista del formato Open CVN JSON	El JSON Schema no puede expresar toda relación conceptual curada ni toda regla semántica del dominio
Importación de CVN XML con trazabilidad y diagnóstico explícito	Mapeo semántico parcial, creciente sobre los casos ya cubiertos; no es un conversor completo
Importación asistida por LLM opcional y validada antes de almacenarse	Depende de un proveedor externo; el resultado no es autoritativo sin revisión
Exportación determinista a LaTeX y PDF	La generación de PDF depende de un motor LaTeX disponible en el entorno

Tabla 7.2: Garantías del sistema frente a limitaciones conocidas.

En conjunto, el sistema desarrollado ofrece garantías fuertes y reproducibles allí donde el proceso es determinista: la generación estructural, la normalización, la resolución de referencias con evidencia concreta, la política semántica y la validación del formato Open CVN JSON. Ofrece garantías parciales y explícitamente documentadas allí donde el propio dominio o el alcance del trabajo lo exigen: la importación de CVN XML, la importación asistida

por LLM y la generación de PDF. Ninguno de estos resultados debe interpretarse como una conversión completa del ecosistema CVN ni como una validación semántica total del dominio curricular; ambas interpretaciones excederían lo que la evidencia reunida en este capítulo permite afirmar. El Capítulo 8 retoma esta distinción para valorar el grado de cumplimiento de los objetivos planteados en el Capítulo 1 y para señalar las líneas de trabajo futuro que se derivan de las limitaciones aquí documentadas.

8. Conclusiones, competencias y trabajo futuro

El Capítulo 1 planteó el problema de partida, la motivación del trabajo y un objetivo general acompañado de diez objetivos específicos. Los capítulos intermedios han desarrollado, capa a capa, la arquitectura Open CVN que responde a ese planteamiento, y el Capítulo 7 ha aportado evidencia reproducible de su comportamiento. El presente capítulo cierra el documento retomando esos objetivos para valorar su grado de cumplimiento, relacionando el trabajo realizado con las competencias de Computación implicadas, y señalando las limitaciones y líneas de trabajo futuro que se derivan de todo lo anterior.

8.1. Resumen del trabajo

Problema y motivación. CVN proporciona a España una norma consolidada para representar currículos académicos e investigadores, pero su uso automatizado sigue siendo costoso: su representación XML oficial y la herramienta asociada no se diseñaron como base directa de aplicaciones abiertas y ligeras, tal y como se planteó en el Capítulo 1 y se analizó con evidencia técnica concreta en el Capítulo 3.

Arquitectura desarrollada. Frente a ese problema, este trabajo construye Open CVN: una arquitectura por capas, presentada en el Capítulo 4, que separa el paquete oficial CVN, los metadatos normalizados, la política semántica, los modelos de dominio, el modelo conceptual y el formato Open CVN JSON. El Capítulo 5 implementa la generación estructural, la normalización y la política semántica que alimentan esa arquitectura, y el Capítulo 6 completa el recorrido con el formato Open CVN JSON, su parser, su validador y una herramienta local que demuestra su uso.

Resultados. El Capítulo 7 evalúa esta arquitectura con una batería de 488 pruebas automatizadas y reproducibles, organizada en ocho niveles alineados con la propia arquitectura,

sin ningún fallo en su última ejecución. Los resultados confirman garantías fuertes allí donde el proceso es determinista (generación estructural, normalización, política semántica y validación del formato Open CVN JSON) y garantías parciales, documentadas de forma explícita, donde el propio dominio o el alcance del trabajo lo exigen (importación de CVN XML e importación asistida por LLM).

8.2. Cumplimiento de los objetivos

El objetivo general planteado en el Capítulo 1 consistía en diseñar e implementar una arquitectura computacional abierta capaz de representar, validar, transformar, almacenar y exportar currículos académicos e investigadores a partir de CVN, sin que la representación XML oficial ni la herramienta oficial condicionaran por completo el modelo interno del sistema. Este objetivo se considera cumplido: la arquitectura por capas presentada en el Capítulo 4 e implementada en los Capítulos 5 y 6 separa de forma explícita el paquete oficial CVN, los metadatos normalizados, el modelo conceptual y el formato Open CVN JSON, y la evaluación del Capítulo 7 confirma que esa separación funciona de forma reproducible.

Únicamente uno de los objetivos específicos se cumple con garantías parciales, y de forma deliberada: la importación desde CVN XML es, por diseño, un mapeo semántico parcial y creciente, y la importación asistida por LLM es opcional y no se considera autoritativa sin revisión, tal y como se discutió con evidencia en el Capítulo 7. El resto de objetivos se apoya en mecanismos deterministas que la batería de pruebas verifica de forma reproducible. La Tabla 8.1 detalla el grado de cumplimiento de cada objetivo específico, en el mismo orden en que se presentaron en el Capítulo 1.

Obj.	Descripción resumida	Grado de cumplimiento
OE1	Análisis del ecosistema CVN y sus artefactos oficiales	Cumplido
OE2	Estudio de alternativas de representación, modelado y validación	Cumplido
OE3	Generación estructural reproducible desde XSD	Cumplido
OE4	Normalización de metadatos CVN	Cumplido
OE5	Definición de reglas semánticas trazables	Cumplido
OE6	Generación de modelos de dominio y artefactos conceptuales	Cumplido
OE7	Definición del formato Open CVN JSON	Cumplido
OE8	Parser, validador, almacenamiento y exportación	Cumplido
OE9	Importación determinista y asistida por LLM	Cumplido con garantías parciales
OE10	Verificación automatizada y flujos reproducibles	Cumplido

Tabla 8.1: Grado de cumplimiento de los objetivos específicos.

8.3. Contribuciones principales

Las contribuciones anunciadas en el Capítulo 1 quedan demostradas, con evidencia concreta, a lo largo del documento.

La primera es el **análisis computacional del ecosistema CVN**: el Capítulo 3 documenta con evidencia concreta las cuatro capas del paquete oficial y sus tres familias auxiliares, junto con las limitaciones técnicas detectadas, en lugar de asumir que dicho paquete puede tratarse como una fuente homogénea y sin fricciones.

La segunda es la **arquitectura reproducible por capas** presentada en el Capítulo 4 e implementada en el Capítulo 5: la separación entre bindings generados, normalización, política semántica, modelos de dominio, modelo conceptual, formato JSON y herramienta local evita que una única representación concentre todas las responsabilidades del sistema.

La tercera es la **trazabilidad entre CVN y Open CVN**: el Capítulo 5 presenta un mecanismo concreto de trazabilidad, con anotaciones no validantes y un recorrido completo y verificable desde un código CVN hasta su definición final en el formato de salida.

La cuarta es el propio **formato Open CVN JSON**, cuya estructura raíz, organización por áreas conceptuales y forma común de las referencias controladas se definen en el Capítulo 6.

La quinta son las **herramientas de validación y uso**: el parser, el validador y la herramienta local de gestión, con su modelo de currículo maestro y versiones derivadas, descritos en el mismo Capítulo 6.

La sexta es la **importación determinista y asistida por LLM**: las vías de importación desde CVN XML y CVN PDF, con el fallback opcional por LLM sujeto a validación posterior, presentadas también en el Capítulo 6.

La séptima y última es la **verificación automatizada**: la batería de 488 pruebas automatizadas, organizada en ocho niveles alineados con la arquitectura, documentada con resultados reproducibles en el Capítulo 7.

Estas contribuciones no sustituyen al ecosistema CVN oficial, sino que proponen una capa complementaria que facilita su tratamiento computacional en contextos donde son necesarias la validación formal, la transformación entre formatos y la gestión automatizada de currículos.

8.4. Competencias desarrolladas

El Capítulo 1 identificó cuatro competencias de la intensificación de Computación trabajadas en este proyecto y anunció que su grado de cumplimiento se retomaría en las conclusiones. La Tabla 8.2 recoge esa evidencia concreta.

Comp.	Evidencia concreta
CM1	El diseño de la arquitectura por capas del Capítulo 4 y la definición del formato Open CVN JSON del Capítulo 6 como artefactos técnicos nuevos derivados de un análisis conceptual del dominio curricular.
CM2	El procesamiento léxico, sintáctico y semántico de los esquemas y documentos CVN a lo largo del pipeline del Capítulo 5: generación estructural desde XSD, normalización de metadatos y política semántica que interpreta y traduce dicha estructura.
CM5	La formalización del conocimiento curricular contenido en el paquete oficial CVN en un modelo conceptual computable y validable, desarrollada en los Capítulos 3 a 6.
CM6	La herramienta local de gestión del Capítulo 6, que presenta información curricular compleja y permite seleccionar, versionar y exportar secciones concretas de un currículo mediante una interfaz de línea de órdenes.

Tabla 8.2: Competencias de Computación y evidencia concreta de cada una.

8.5. Limitaciones y trabajo futuro

Los capítulos anteriores han documentado limitaciones concretas en distintos puntos del sistema, sin ocultarlas ni presentarlas como defectos de implementación cuando, en realidad, proceden del propio paquete oficial o de una decisión deliberada de alcance, tal y como ya se distinguió en el Capítulo 7. Esta sección retoma cada una de ellas con el detalle suficiente para justificar la línea de trabajo futuro que le corresponde, antes de consolidarlas en una tabla de referencia rápida.

La primera limitación son las **inconsistencias entre artefactos del paquete oficial CVN**, como la discrepancia puntual entre el XML canónico del modelo en árbol y su esquema de validación, o la deriva de empaquetado entre las familias auxiliares descrita en el Capítulo 3. Esta limitación no es atribuible al sistema desarrollado, sino al propio material fuente, por lo que el trabajo futuro correspondiente no puede consistir en corregirla, sino en revisarla si el paquete oficial se actualiza y documentar cualquier discrepancia nueva con el mismo criterio de evidencia ya aplicado.

La segunda es la **expresividad limitada de algunos bindings estructurales generados**: construcciones como el mecanismo *choice* de XSD, que no siempre se traduce en una exclusión mutua real, o los atributos generados con un tipo demasiado genérico, tal y como se analizó en el Capítulo 3. Estas limitaciones se confinan de forma deliberada a la capa estructural generada y no se propagan al formato Open CVN JSON, pero el trabajo futuro puede ampliar las validaciones de dominio que compensan esta pérdida de expresividad fuera de dicha capa.

La tercera son las **referencias auxiliares no resueltas con el paquete actual**, de las que el caso de `CVN_AGENCY_C` es el ejemplo verificado en el Capítulo 5: una tabla citada por el manual sin equivalente localizable en el resto del paquete. El sistema conserva esta referencia como no resuelta en lugar de forzar una interpretación no justificada, y el trabajo futuro solo puede mejorar la cobertura de tablas auxiliares si aparece evidencia oficial adicional que hoy no existe.

La cuarta es la **importación de CVN XML semánticamente parcial**: un mapeo creciente sobre los casos ya cubiertos, y no un conversor completo de cualquier documento CVN XML real, según se discutió con evidencia en el Capítulo 7. A diferencia de las dos limitaciones anteriores, esta sí admite una línea de trabajo futuro directa y sustancial: completar el mapeo semántico sobre más casos reales y evaluar el sistema con currículos reales, en lugar de únicamente con los ejemplos sintéticos empleados en este trabajo.

La quinta es que el **JSON Schema no puede expresar invariantes entre atributos**,

porque un esquema de validación declarativo, igual que un diagrama de clases UML, solo puede describir la estructura, el tipo y la cardinalidad de un atributo aislado, no reglas que combinen varios atributos entre sí. El modelo conceptual actual ya contiene, sin formalizar, patrones de este tipo: un campo de valor controlado seguido de un campo de texto libre para el caso "otros", donde la regla de negocio implícita exige que ese texto libre sea obligatorio únicamente cuando se elige dicha opción; o un par de fechas de inicio y de fin dentro de una misma entidad, donde se espera que la primera preceda a la segunda. Ninguna de estas reglas está implementada actualmente ni en los modelos generados ni en los diagramas conceptuales, por lo que el trabajo futuro correspondiente, estudiar la incorporación de restricciones OCL sobre el modelo conceptual, constituiría una extensión deliberada de dicho modelo, y no la transcripción de una validación ya existente.

La sexta y última es la **generación de PDF dependiente de un motor LaTeX externo**, ya señalada en el Capítulo 6: la exportación aprovecha una cadena de herramientas LaTeX ya madura en lugar de reimplementar un renderizador de PDF propio, a costa de exigir dicho motor en el entorno de ejecución. El trabajo futuro puede explorar mecanismos de generación adicionales que reduzcan esta dependencia sin renunciar a la calidad tipográfica de la exportación actual.

Además de estas seis líneas, ligadas cada una a una limitación concreta, el trabajo admite otras extensiones de alcance no derivadas de una limitación específica, sino de una ampliación natural del sistema desarrollado: desarrollar una interfaz gráfica si se considera necesaria más allá de la herramienta de línea de órdenes actual, estudiar extensiones semánticas como JSON-LD para facilitar la integración con datos enlazados, e investigar la integración con fuentes institucionales de información curricular.

La Tabla 8.3 consolida, a modo de referencia rápida, las seis limitaciones ya explicadas junto con la línea de trabajo futuro que se deriva de cada una.

Limitación	Trabajo futuro asociado
Inconsistencias entre artefactos del paquete oficial CVN (discrepancia XML/XSD, deriva de empaquetado)	Revisar estas inconsistencias si el paquete oficial se actualiza, documentando cualquier discrepancia nueva con el mismo criterio de evidencia
Expresividad limitada de algunos bindings estructurales generados (*choice* sin exclusión mutua, tipos genéricos)	Ampliar las validaciones de dominio fuera de la capa estructural generada
Referencias auxiliares no resueltas con el paquete actual	Mejorar la cobertura de tablas auxiliares si aparece evidencia oficial adicional
Importación de CVN XML semánticamente parcial	Completar el mapeo semántico sobre más casos reales y evaluar el sistema con currículos reales
JSON Schema sin capacidad de expresar invariantes entre atributos	Estudiar la incorporación de restricciones OCL sobre el modelo conceptual
Generación de PDF dependiente de un motor LaTeX externo	Explorar mecanismos de generación adicionales que reduzcan esta dependencia

Tabla 8.3: Limitaciones documentadas y trabajo futuro asociado.

8.6. Conclusiones

El resumen, el cumplimiento de objetivos y las contribuciones ya presentados en este capítulo permiten cerrar el documento con una valoración conjunta, en lugar de con la simple repetición de lo ya expuesto. El objetivo general se considera cumplido y, con él, los diez objetivos específicos que lo desarrollaban, con la única salvedad deliberada de la importación asistida por LLM, cuyo carácter opcional y no autoritativo es una decisión de diseño y no una carencia. Esta conclusión no se apoya en una afirmación sin respaldo: el Capítulo 7 la sostiene con una batería de 488 pruebas automatizadas y reproducibles, organizada según las mismas capas que estructuran el documento, lo que permite sostener que las garantías descritas no son solo argumentativas, sino verificables por cualquier persona que ejecute el mismo comando sobre el mismo repositorio.

Las limitaciones recogidas en este capítulo no comprometen esta conclusión: delimitan, con precisión y sin ocultamiento, hasta dónde llega la propuesta actual y qué preguntas quedan abiertas para su continuación. Algunas de ellas son inherentes al propio paquete oficial CVN y quedan fuera del control del sistema desarrollado; otras son decisiones deliberadas de alcance, propias de un Trabajo Fin de Grado y no de un producto cerrado. En ambos casos, el sistema las documenta en lugar de disimularlas, lo que constituye en sí mismo una forma de rigor

técnico.

En conjunto, este trabajo demuestra que es posible construir, a partir del ecosistema CVN, una arquitectura computacional abierta que separa la interoperabilidad estructural del significado curricular, conserva trazabilidad verificable hacia sus fuentes oficiales y ofrece garantías reproducibles allí donde el proceso es determinista. Esa arquitectura, más que el formato JSON o la herramienta local que la demuestran, es la aportación principal de este Trabajo Fin de Grado a la intensificación de Computación en la que se enmarca.

Referencia bibliográfica

[Astral, 2026] Astral (2026). uv: An extremely fast python package and project manager. https://docs.astral.sh/uv/. Accedido: 2026-07-23.

[BESSER, 2026] BESSER (2026). Pydantic generator - BESSER documentation. https://besser.readthedocs.io/en/latest/generators/pydantic.html. Accedido: 2026-02-25.

[Deusto HERCULES, 2026a] Deusto HERCULES (2026a). CVN. https://github.com/deustohercules/CVN. Accedido: 2026-02-16.

[Deusto HERCULES, 2026b] Deusto HERCULES (2026b). ROH-1. https://github.com/deustohercules/ROH-1. Accedido: 2026-02-16.

[euroCRIS, 2026] euroCRIS (2026). Main features of CERIF. https://eurocris.org/services/main-features-cerif. Accedido: 2026-02-16.

[FECYT, 2026] FECYT (2026). Norma CVN. https://cvn.fecyt.es/. Accedido: 2026-02-16.

[GitHub, 2026] GitHub (2026). GitHub Actions documentation. https://docs.github.com/en/actions. Accedido: 2026-07-23.

[JSON Schema, 2026] JSON Schema (2026). What is JSON Schema? https://json-schema.org/overview/what-is-jsonschema. Accedido: 2026-07-22.

[JSON.org, 2026] JSON.org (2026). Introduccion a JSON. https://json.org/json-es.html. Accedido: 2026-02-19.

[LinkML, 2026] LinkML (2026). Linked data modeling language (LinkML). https://linkml.io/. Accedido: 2026-02-23.

[Object Management Group, 2026a] Object Management Group (2026a). Object constraint language (OCL). https://www.omg.org/spec/OCL/. Accedido: 2026-02-25.

[Object Management Group, 2026b] Object Management Group (2026b). Unified modeling language (UML). https://www.omg.org/uml/. Accedido: 2026-02-25.

[ORCID, 2026] ORCID (2026). About ORCID. https://info.orcid.org/what-is-orcid/. Accedido: 2026-07-22.

[Pallets Projects, 2026] Pallets Projects (2026). Jinja documentation. https://jinja.palletsprojects.com/. Accedido: 2026-07-23.

[Proyecto HERCULES, 2026] Proyecto HERCULES (2026). Research ontology for hercules (ROH). https://deustohercules.github.io/roh/roh/index.html. Accedido: 2026-02-16.

[Pydantic, 2026] Pydantic (2026). Pydantic documentation. https://docs.pydantic.dev/latest/. Accedido: 2026-07-22.

[pytest, 2026] pytest (2026). pytest documentation. https://docs.pytest.org/en/stable/. Accedido: 2026-07-23.

[RO-Crate Community, 2024] RO-Crate Community (2024). Research object crate (RO-Crate). https://www.researchobject.org/ro-crate/. Accedido: 2026-07-22.

[SQLite, 2026] SQLite (2026). SQLite home page. https://www.sqlite.org/index.html. Accedido: 2026-07-23.

[Universidad de Murcia, 2026] Universidad de Murcia (2026). Ontologias - proyecto HERCULES. https://www.um.es/web/hercules/ontologias. Accedido: 2026-02-16.

[VIVO Project, 2026] VIVO Project (2026). VIVO: Connect, share, discover. https://www.vivoweb.org/. Accedido: 2026-02-16.

[World Wide Web Consortium, 2008] World Wide Web Consortium (2008). Extensible markup language (XML) 1.0 (fifth edition). https://www.w3.org/TR/xml/. W3C Recommendation, accedido: 2026-07-22.

[World Wide Web Consortium, 2012] World Wide Web Consortium (2012). W3C XML Schema Definition Language (XSD) 1.1 part 1: Structures. https://www.w3.org/TR/xmlschema11-1/. W3C Recommendation, accedido: 2026-07-22.

[World Wide Web Consortium, 2020] World Wide Web Consortium (2020). JSON-LD 1.1: A JSON-based serialization for linked data. https://www.w3.org/TR/json-ld11/. W3C Recommendation, accedido: 2026-07-22.

[xsdata, 2026] xsdata (2026). xsdata documentation. https://xsdata.readthedocs.io/en/latest/. Accedido: 2026-07-23.

[YAML.org, 2026] YAML.org (2026). YAML ain't markup language. https://yaml.org/. Accedido: 2026-02-23.

Declaración de uso de Inteligencia Artificial

En la elaboración de este Trabajo Fin de Grado se han empleado modelos de Inteligencia Artificial Generativa de OpenAI y de Anthropic como apoyo en la búsqueda de documentación técnica de las herramientas y tecnologías utilizadas, en la redacción académica de la memoria y en la revisión de la coherencia entre sus capítulos.

Todo resultado generado por un modelo de Inteligencia Artificial Generativa ha sido revisado y, en caso de ser necesario, corregido por el autor. El resto de aspectos del trabajo, incluyendo el diseño de la arquitectura, la implementación del sistema, la generación de los resultados y su interpretación, son responsabilidad exclusiva del autor.

Anexos

Los siguientes anexos recogen información de referencia complementaria al cuerpo principal de la memoria: demasiado extensa para incluirse en los capítulos anteriores, pero necesaria para reproducir el entorno de trabajo, consultar el detalle del formato Open CVN JSON, trazar el desarrollo del proyecto por fases técnicas y acceder al repositorio del código fuente.

Anexo A: Instalación, entorno y comandos principales

Este anexo recoge, de forma reproducible, los pasos necesarios para preparar el entorno de desarrollo, regenerar los artefactos derivados del paquete oficial CVN y ejecutar la herramienta local de gestión descrita en el Capítulo 6. No introduce ninguna decisión de diseño nueva: es una referencia operativa que complementa, con la sintaxis exacta de los comandos, la Tabla 6.1.

A.1. Prerrequisitos y configuración del entorno

El proyecto requiere `uv` como gestor de entorno y dependencias, y Python 3.14. El Código A.1 resume la secuencia completa de preparación: fijar la versión de Python, sincronizar los grupos de dependencias necesarios e instalar el proyecto en modo editable, requisito para que el código bajo `src/` sea importable durante la ejecución de pruebas y de la herramienta local.

El grupo de dependencias `codegen` incluye `xsdata` y `xsdata-pydantic`, empleados por la generación estructural descrita en la Sección 5.1; el grupo `testing` incluye `pytest` y `pytest-xdist`, empleados por la batería de pruebas evaluada en el Capítulo 7.

```
# Fijar la version de Python del proyecto (opcional si ya esta activa)
uv python pin 3.14

# Sincronizar dependencias de generacion estructural y de pruebas
uv sync --group codegen --group testing

# Instalar el proyecto en modo editable
uv pip install -e .

# Verificar que la herramienta local esta disponible
uv run open-cvn --help
```

Código A.1: Preparación del entorno de desarrollo.

A.2. Regeneración de artefactos y verificación

Los artefactos generados bajo `src/generated/` no se editan a mano: se regeneran a partir de los esquemas XSD oficiales cuando estos cambian. El Código A.2 recoge el comando que regenera los tres objetivos estructurales (paquete CVN, manual de especificación y modelo en árbol) y la verificación completa mediante la batería de pruebas automatizadas, la misma que documenta el Capítulo 7.

```
# Regenerar los tres objetivos estructurales
uv run python -m cvn_codegen.xsdata_runner all

# Ejecutar la bateria completa de pruebas en paralelo
uv run pytest -n auto tests
```

Código A.2: Regeneración de artefactos estructurales y verificación por pruebas.

A.3. Recorrido completo con la herramienta local

El Código A.3 muestra un recorrido de extremo a extremo con la herramienta local `open-cvn`: inicialización de un almacén local, importación de un documento Open CVN JSON como currículo maestro, creación de una versión derivada, exploración y exclusión de una entrada, y exportación a Open CVN JSON y a LaTeX. Cada subcomando se corresponde con una fila de la Tabla 6.1.

```
STORE=/tmp/open-cvn-demo.sqlite

# 1. Inicializar el almacen local
uv run open-cvn store init --path $STORE

# 2. Importar un documento Open CVN JSON como maestro
uv run open-cvn json import examples/open_cvn/research_entry.json \
  --store $STORE --as-master

# 3. Crear una version derivada
uv run open-cvn versions derive public --store $STORE

# 4. Explorar las secciones de la version derivada
uv run open-cvn versions sections public --store $STORE

# 5. Excluir una entrada concreta de la version derivada
uv run open-cvn versions exclude public /curriculum/research/0 \
  --store $STORE

# 6. Exportar la version derivada a Open CVN JSON
uv run open-cvn json export /tmp/public.json \
  --store $STORE --version public

# 7. Exportar la version derivada a LaTeX
uv run open-cvn latex export /tmp/public.tex \
  --store $STORE --version public
```

Código A.3: Recorrido completo de importación, versionado y exportación con la herramienta local.

La generación de PDF a partir de la exportación LaTeX, mediante `open-cvn pdf generate`, es opcional y depende de que exista un motor LaTeX disponible en el sistema (`latexmk` o `pdflatex`); en su ausencia, la herramienta lo reporta de forma explícita en lugar de fallar de forma silenciosa. La importación de currículos desde PDF mediante modelos de lenguaje de gran tamaño exige, además, activar explícitamente el indicador `--allow-external-llm`, precisamente porque un PDF puede contener datos personales; este mecanismo se describe con más detalle en la Sección 6.3 y en la Sección 7.4.

Anexo B: Estructura completa del formato Open CVN JSON

La Sección 6.1 introdujo la estructura raíz del formato Open CVN JSON y su forma común para valores de referencia controlada. Este anexo completa esa descripción con el detalle de cada bloque, tal como está declarado en el artefacto canónico `schemas/open_cvn.schema.json` generado por el pipeline descrito en el Capítulo 5. El objetivo es ofrecer una referencia legible por personas de la forma del formato, no sustituir el esquema JSON como fuente de verdad para la validación automática.

B.1. Bloque `metadata`

El bloque `metadata` describe el documento, no su contenido curricular, y contiene tres campos:

- `language`: idioma del contenido curricular, expresado como código de idioma.
- `policy`: objeto con `name` y `version`, que identifica la política semántica aplicada durante la generación del documento, en el sentido descrito en la Sección 5.4.
- `source`: objeto con `format`, que identifica el origen del documento cuando procede de un proceso de importación.

B.2. Bloque `curriculum`

El bloque `curriculum` organiza la información por área conceptual. El campo `identity` es un único objeto que describe a la persona titular del currículo; a diferencia del resto de cam-

pos, el esquema canónico lo declara como un objeto abierto, sin una lista cerrada de propiedades, porque la identidad agrupa datos de naturaleza heterogénea (nombre, identificadores oficiales, sexo administrativo, trazas) que no comparten una única forma repetida.

Los cinco campos restantes son listas de entradas repetidas: `education`, `research`, `professional_experience`, `achievements` y `other`. Cada entrada de estas listas comparte una misma envoltura, resumida en la Tabla B.1.

Campo	Contenido
`id`	Identificador local opcional de la entrada dentro del documento
`type`	Identificador conceptual estable del tipo de entrada, independiente del código CVN de origen
`data`	Objeto con los campos propios de la entrada, según su `type`
`extensions`	Objeto opcional para metadatos específicos de una herramienta o proyecto concreto
`trace`	Bloque opcional de trazabilidad hacia el paquete oficial CVN

Tabla B.1: Envoltura común de las entradas de las listas curriculares.

Cuando está presente, el bloque `trace` puede incluir los códigos CVN de origen (`cvn_codes`), las rutas XML de las que procede (`xml_paths`), el nivel de confianza de la extracción (`confidence`), y campos adicionales de apoyo a la trazabilidad técnica: forma del dominio, tabla de referencia del manual funcional, tipo de referencia semántica, patrón de serialización, y los artefactos y ficheros de origen concretos. Este mismo mecanismo de trazabilidad es el que permite, en la dirección inversa, relacionar cada campo del formato Open CVN con el código CVN del que procede, tal como se describe en la Sección 5.2.

B.3. Valores de referencia controlada

Los campos que remiten a una tabla de valores controlados comparten la forma introducida en la Sección 6.1: un objeto con `code`, `label`, `source` y, cuando procede, `raw_value` o `uri`. El esquema canónico distingue dos tratamientos posibles para esta forma, ya contrastados con evidencia concreta en la Sección 5.4:

- cuando la tabla de valores está cerrada y tiene evidencia suficiente, el campo `code` se emite como una enumeración cerrada en el esquema, como ocurre con `CVN_SEX_A`;
- cuando la tabla no reúne evidencia suficiente para cerrarla sin perder valores válidos,

el campo `code` permanece abierto, como ocurre con `CVN_ENTITY_TYPE`.

B.4. Definiciones del esquema

El esquema canónico declara sus definiciones reutilizables bajo `$defs`. La Tabla B.2 resume su distribución por área conceptual, verificada directamente sobre el artefacto generado.

Área	Definiciones	Contenido
`education`	30	formación académica e impartida
`research`	47	actividad investigadora
`professional_experience`	3	experiencia profesional
`achievements`	2	méritos y reconocimientos
`other`	20	entradas sin área conceptual específica
`identity`	1	identidad de la persona titular
`core`	1	forma raíz del currículo
vocabularios	74	valores de referencia controlada
Total	**182**	

Tabla B.2: Distribución de las definiciones del esquema Open CVN JSON por área conceptual.

Cada definición conserva, cuando existe evidencia, una traza no validante hacia su origen mediante extensiones `x-open-cvn-*`: código CVN de origen, rutas XML, forma del dominio conceptual, elegibilidad como enumeración, referencia de origen y nivel de confianza. Estas extensiones no participan en la validación JSON Schema en sentido estricto, pero permiten auditar cada definición hasta su fuente oficial, en línea con el principio de trazabilidad que atraviesa todo el trabajo.

Los artefactos citados en este anexo, el esquema canónico `schemas/open_cvn.schema.json` y los documentos de ejemplo bajo `examples/open_cvn/`, son reproducibles a partir del código fuente del proyecto siguiendo el procedimiento descrito en el Anexo A.

Anexo C: Desarrollo del proyecto por fases técnicas

Los capítulos principales de esta memoria describen la arquitectura de Open CVN y sus resultados de forma independiente del proceso concreto que llevó a construirla. Este anexo es una excepción deliberada a esa regla: su propósito específico es documentar la trazabilidad del desarrollo, por lo que identifica de forma explícita las unidades de trabajo internas (issues y correcciones puntuales) que el repositorio utilizó para planificar y registrar cada fase. El desarrollo se organizó en tres fases técnicas sucesivas, cada una cerrada con su propia batería de pruebas antes de iniciar la siguiente.

C.1. Fase 1: generación estructural y normalización

Esta fase construyó la base de todo el trabajo posterior: la traducción reproducible del paquete oficial CVN a artefactos Python y la normalización de sus metadatos dispersos, descritas en las Secciones 5.1 y 5.2. La Tabla C.1 resume sus unidades de trabajo.

Issue	Aportación
#11	Infraestructura de proyecto para la generación de código
#12	Bindings Pydantic estructurales a partir de los esquemas XSD de CVN
#13	Normalización de los metadatos del manual de especificación y del modelo en árbol
#14	Definición de las reglas de mapeo semántico y de la política de resolución de conflictos
#15	Implementación del generador de modelos de dominio Pydantic
#16	Pruebas automatizadas del pipeline de generación
#17	Documentación y automatización del flujo de trabajo completo
#25	Pipeline de integración continua para la validación de contribuciones

Tabla C.1: Unidades de trabajo de la Fase 1, agrupadas bajo el epic de automatización de la traducción CVN XML/XSD.

C.2. Fase 2: modelo conceptual, formato Open CVN y parser

Esta fase construyó, sobre la base anterior, el modelo conceptual agnóstico, el formato de intercambio Open CVN JSON y el parser y validador públicos, descritos en las Secciones 5.5 y 5.6 y en el Capítulo 6. La Tabla C.2 resume sus unidades de trabajo.

Issue	Aportación
#42	Investigación de opciones para derivar diagramas UML de los modelos Pydantic
#43	Definición de la capa de extracción del modelo conceptual agnóstico
#44	Generación de diagramas UML o asimilables a UML
#45	Generación de JSON Schema a partir de los modelos de dominio
#46	Definición del formato canónico Open CVN JSON
#47	Definición del contrato unificado de parser y validador
#48	Extracción de XML embebido desde CVN PDF
#49	Validación de la importación de XML y JSON
#50	Pruebas y documentación del flujo de trabajo del parser

Tabla C.2: Unidades de trabajo de la Fase 2, agrupadas bajo el epic de esquema agnóstico, JSON Schema y parser.

C.3. Fase 3: aplicación de gestión local

Esta fase construyó la herramienta local de línea de órdenes que demuestra el uso completo de Open CVN, descrita en la Sección 6.4, incluyendo los mecanismos de importación semántica y asistida por modelos de lenguaje que se evalúan con garantías parciales en el Capítulo 7. La Tabla C.3 resume sus unidades de trabajo.

Issue	Aportación
#61	Definición del alcance del MVP de la aplicación y de la interfaz de línea de órdenes
#62	Implementación del almacenamiento local con SQLite
#63	Implementación de versiones maestra y derivadas del currículo
#64	Implementación del flujo de importación y exportación de Open CVN JSON
#65	Implementación de la edición y selección de currículo en el MVP
#66	Implementación de la exportación a LaTeX desde Open CVN
#67	Implementación de la generación de PDF y de la entrega de la vista previa
#68	Pruebas y documentación del MVP de la aplicación
#69	Spike de importación de PDF asistida por modelos de lenguaje
#70	Importación semántica de CVN XML a Open CVN JSON
#71	Endurecimiento de limitaciones y documentación

Tabla C.3: Unidades de trabajo de la Fase 3, agrupadas bajo el epic de la aplicación de gestión de currículos.

C.4. Correcciones de alcance

Además de las tres fases planificadas, el desarrollo registró ocho correcciones puntuales de alcance, aplicadas cuando la implementación reveló que una fuente auxiliar del paquete oficial CVN (tablas de referencia, catálogos, convenciones del manual funcional) no encajaba con la planificación inicial. Estas correcciones no añaden funcionalidad nueva: ajustan decisiones previas a la vista de evidencia concreta encontrada durante la implementación, y se documentan aquí porque forman parte legítima de la trazabilidad del proceso. La Tabla C.4 las resume.

La concentración de correcciones en torno a las fuentes auxiliares del paquete oficial CVN (tablas de referencia y catálogos) no es casual: es la misma tensión entre heterogeneidad documental y necesidad de trazabilidad que motiva la política semántica conservadora descrita

Corrección	Motivo
#1	Actualización de la convención de registro del runner de generación
#2	Alineación del punto de entrada humano del proyecto con el protocolo de actualización de documentación
#3	Ampliación de la documentación del paquete fuente oficial de CVN
#4	Corrección del alcance estructural para artefactos auxiliares del paquete fuente
#5	Capa de resolución de normalización para fuentes de referencia auxiliares
#6	Realineación de la planificación para la integración semántica de catálogos auxiliares
#7	Evaluación dinámica de la elegibilidad de enumeración en tablas de referencia
#8	Trazabilidad de tipos envoltorio en el traspaso de datos normalizados

Tabla C.4: Correcciones de alcance aplicadas durante el desarrollo.

en la Sección 5.4, y confirma en la práctica que esa cautela de diseño estaba justificada.

Anexo D: Repositorio del proyecto y guía de uso

El código fuente completo de Open CVN, junto con esta misma memoria, está disponible públicamente en el siguiente repositorio:

`https://github.com/holt00/TFG-open-cvn-schema`

Este anexo describe la organización general del repositorio y ofrece una guía de orientación para quien quiera consultarlo o ejecutarlo, sin repetir el detalle de comandos ya recogido en el Anexo A.

Estructura del repositorio

El repositorio se organiza en un pequeño número de directorios de nivel superior, cada uno con una responsabilidad clara:

- `src/`: código fuente del proyecto, organizado según la arquitectura por capas descrita en el Capítulo 4: bindings estructurales generados, lógica de generación y normalización mantenida a mano, modelos de dominio, el contrato público de análisis y validación, y la herramienta local de gestión.
- `schemas/`: artefacto JSON Schema canónico del formato Open CVN, detallado en el Anexo B.
- `examples/`: documentos Open CVN JSON de ejemplo, citados a lo largo de esta memoria.
- `tests/`: batería de pruebas automatizadas evaluada en el Capítulo 7.

- `docs/`: documentación técnica del proyecto, organizada por tema (arquitectura, pipeline, formato, desarrollo) y el registro de planificación por issues y correcciones puntuales resumido en el Anexo C. El propio código fuente de esta memoria se encuentra en `docs/memoria/`.

Cómo orientarse en el repositorio

Para quien no haya seguido el desarrollo del proyecto, el punto de entrada recomendado es el fichero `README.md` en la raíz del repositorio, que resume el objetivo del proyecto y su alcance actual. El fichero `PROJECT_GUIDE.md` amplía esa descripción con un mapa completo de la documentación técnica disponible bajo `docs/`, incluida la correspondencia entre cada capítulo de esta memoria y su documentación técnica de referencia en el repositorio.

Para preparar el entorno, regenerar los artefactos y ejecutar la batería de pruebas o la herramienta local, el Anexo A recoge los comandos exactos y un recorrido completo de ejemplo.